DOCUMENTO DE ARQUITECTURA, REQUISITOS Y DISEÑO DE SOFTWARE



BANCO DE PREGUNTAS SABER PRO

Presentado por:

Britney Benítez Ordoñez

Primera entrega del proyecto de curso · Primer corte · 2026.2

Profesor:

Paola Andrea Bedoya Toro

Julio Ariel Hurtado

Universidad del Cauca

Facultad de Ingeniería Electrónica y Telecomunicaciones

Departamento de Sistemas

Programa de Ingeniería de Sistemas · Ingeniería de Software II

Popayán, octubre de 2026

Contenido

1.- Introducción 1
1.1.- Problema y justificación 1
1.2.- Objetivo general 1
1.3.- Objetivos específicos 1
1.4.- Alcance y exclusiones 1
2.- Actores y términos 2
2.1.- Glosario 2
2.2.- Reglas de negocio transversales 3
3.- Especificación de requisitos 4
3.1.- Requisitos funcionales (RF) 4
3.2.- Requisitos no funcionales (RNF) 5
3.3.- Restricciones y supuestos 7
4.- Historias de usuario y especificaciones 7
4.1.- HU-01 · Registrar pregunta Saber Pro 7
4.1.1.- Flujo principal del caso de uso 8
4.1.2.- Flujos alternativos y excepciones 8
4.1.3.- Criterios de aceptación verificables 8
4.1.4.- Componentes y evidencia de verificación 8
4.1.5.- Definition of Ready (DoR) — verificación previa al desarrollo 9
4.1.6.- Definition of Done (DoD) — condición de cierre 9
4.2.- HU-02 · Enviar borrador a revisión 9
4.2.1.- Flujo principal del caso de uso 10
4.2.2.- Flujos alternativos y excepciones 10
4.2.3.- Criterios de aceptación verificables 10
4.2.4.- Componentes y evidencia de verificación 11
4.2.5.- Definition of Ready (DoR) — verificación previa al desarrollo 11
4.2.6.- Definition of Done (DoD) — condición de cierre 11
4.3.- HU-03 · Consultar y filtrar mis preguntas 11
4.3.1.- Flujo principal del caso de uso 12
4.3.2.- Flujos alternativos y excepciones 12
4.3.3.- Criterios de aceptación verificables 12
4.3.4.- Componentes y evidencia de verificación 13
4.3.5.- Definition of Ready (DoR) — verificación previa al desarrollo 13
4.3.6.- Definition of Done (DoD) — condición de cierre 13
4.4.- HU-04 · Asignar y notificar revisores 13
4.4.1.- Flujo principal del caso de uso 14
4.4.2.- Flujos alternativos y excepciones 14
4.4.3.- Criterios de aceptación verificables 14
4.4.4.- Componentes y evidencia de verificación 15
4.4.5.- Definition of Ready (DoR) — verificación previa al desarrollo 15
4.4.6.- Definition of Done (DoD) — condición de cierre 15
5.- Modelado de casos de uso UML 17
5.1.- Catálogo de casos de uso 17
5.2.- Precondiciones y poscondiciones por caso 18
5.3.- Diagrama de actividad y secuencia 19
6.- Arquitectura y diseño con modelo C4 21
6.1.- Principios y decisiones arquitectónicas 21
6.2.- C4 nivel 1: contexto 22
6.3.- C4 nivel 2: contenedores 23

6.4.- C4 nivel 3: componentes 24
6.5.- Mapa de implementación y dependencias 24
7.- Modelo UML, datos y estados 26
7.1.- Diagrama UML de clases representativas 26
7.2.- Modelo de datos persistentes 27
7.3.- Diagramas de estados 28
7.4.- Invariantes de persistencia 28
8.- Patrones de diseño y SOLID 28
8.1.- Ejemplo concreto: decorador de notificación 29
8.2.- Responsabilidad de la sesión demostrativa 30
9.- Escenarios de calidad y medición 30
9.1.- Escenario obligatorio de modificabilidad 30
9.2.- Otros escenarios de calidad (propuestos) 30
9.3.- Plan de verificación de atributos de calidad 31
10.- Prototipos y evaluación de usabilidad 32
10.1.- Inventario de pantallas a evidenciar 32
10.2.- Protocolo de prueba de usabilidad 33
10.3.- Instrumento de recolección 34
10.3.1. Simulación exploratoria de experiencia de usuario 35
10.4.- Hallazgos y mejoras 36
11.- Pruebas y matriz de trazabilidad 37
11.1.- Evidencia técnica de pruebas 37
11.2.- Casos de prueba funcionales mínimos 38
11.3.- Trazabilidad HU-RF-CU-prueba 39
12.- Gestión ágil del proyecto y contribuciones 40
12.1.- Backlog académico y seguimiento de los sprints 40
12.2.- Evidencia de Jira y trazabilidad de Sprint 1 40
12.3.- Contribuciones verificables de la desarrolladora 41
13.- Guion de sustentación (12 minutos) 42
13.1.- Secuencia de demostración funcional sugerida 43
14.- Instalación, operación y seguridad de entrega 43
14.1.- Entorno de ejecución 43
14.2.- Operaciones 44
14.3.- Seguridad de correo y política de entrega 44
14.4.- Control de versiones y ramas 44
15.- Bibliografía, fuentes y enlaces 44
15.1.- Fuentes y enlaces 44
Bibliografía 44

Lista de figuras

Figura 5.1. Casos de uso vinculados a los perfiles demostrativos. El docente es destinatario de correo, no un usuario autenticado. 17
Figura 5.2. Flujo de actividad de una nueva asignación y su notificación (las ramas no exitosas conservan la integridad). 19
Figura 5.3. Secuencia simplificada de persistencia y notificación. En los fallos, el decorador registra error seguro.20
Figura 6.1. Contexto: actores directos, destinatario de notificación y servicio SMTP externo. 22
Figura 6.2. Despliegue real: una JVM Java Swing y un archivo SQLite en el computador del usuario. 23
Figura 6.3. Componentes principales de las tres capas dentro del monolito. 24
Figura 7.1. Entidades de dominio: pregunta, revisor, asignación y notificaciones. 26
Figura 7.2. Contratos y dependencias de servicios y notificación (vista simplificada). 27
Figura 7.3. Esquema lógico SQLite y claves de relación. Los datos de usuario son IDs demostrativos. 27
Figura 7.4. Transiciones implementadas: preguntas y notificaciones. No se documentan como existentes estados de evaluación no implementados. 28
Figura 10.1. Captura del proyecto en Jira y bocetos referenciales de las interfaces HU-01 y HU-04. 32
Figura 12.1. Vista real de Jira: historia HU-01 y cinco subtareas vinculadas en la pestaña Lista. 41

Lista de tablas

Tabla 1.1. Alcance de la primera iteración 2
Tabla 2.1. Actores y responsabilidades 2
Tabla 2.2. Glosario de términos 3
Tabla 3.1. Catálogo de requisitos funcionales 5
Tabla 3.2. Matriz de requisitos no funcionales 6
Tabla 4.1. Ficha de HU-01 7
Tabla 4.2. Ficha de HU-02 10
Tabla 4.3. Ficha de HU-03 12
Tabla 4.4. Ficha de HU-04 14
Tabla 5.1. Catálogo de casos de uso 17
Tabla 5.2. Precondiciones y poscondiciones 18
Tabla 6.1. Decisiones arquitectónicas 22
Tabla 6.2. Mapa de implementación por capas 25
Tabla 8.1. Patrones y principios aplicados 29
Tabla 9.1. Escenario de modificabilidad 30
Tabla 9.2. Escenarios complementarios de calidad 31
Tabla 10.1. Inventario de pantallas a evidenciar 33
Tabla 10.2. Tareas del test de usabilidad 34
Tabla 10.3. Registro individual de tareas completadas en autoevaluación 34
Tabla 10.4. Aspectos de autoevaluación y mejoras propuestas 36
Tabla 11.1. Evidencias técnicas de prueba 37
Tabla 11.2. Casos de prueba funcionales 39
Tabla 11.3. Trazabilidad de historias y pruebas 39
Tabla 12.1. Backlog académico y seguimiento de las cuatro historias 40
Tabla 12.2. Contribuciones individuales verificables 42
Tabla 13.1. Guion de sustentación 43
Tabla 14.1. Dependencias de ejecución 44

Lista de siglas

C4 — Modelo de descripción arquitectónica de contexto, contenedores, componentes y código.
DoD — Definition of Done (definición de terminado).
DoR — Definition of Ready (definición de preparado).
HU — Historia de usuario.
JDK — Java Development Kit.
JVM — Java Virtual Machine.
MVC — Modelo–Vista–Controlador.
RF — Requisito funcional.
RNF — Requisito no funcional.
SMTP — Simple Mail Transfer Protocol.
SOLID — Principios de diseño orientado a objetos.
SQL — Structured Query Language.
TLS — Transport Layer Security.
UML — Unified Modeling Language.

1.- Introducción

El Banco de Preguntas Saber Pro es una aplicación de escritorio en Java diseñada para apoyar el registro académico de preguntas de selección múltiple con única respuesta, el envío de borradores a revisión y la asignación de docentes revisores. La primera iteración implementa las cuatro historias de usuario especificadas por la guía del primer corte de Ingeniería de Software II de la Universidad del Cauca.

1.1.- Problema y justificación

Una gestión manual de preguntas dificulta la validación uniforme de los campos, la trazabilidad de la revisión y la coordinación entre autores y docentes revisores. El sistema centraliza estas operaciones en una misma instalación, valida datos estructurales y registra tanto las asignaciones como los resultados del intento de notificación al revisor.

1.2.- Objetivo general

Implementar un banco académico de preguntas de escritorio que permita el registro, cambio de estado, consulta y asignación de revisores, aplicando una arquitectura monolítica de tres capas, MVC, principios SOLID, patrones de diseño y pruebas automatizadas.

1.3.- Objetivos específicos

- Permitir el registro y validación de preguntas con los campos indicados en la guía del curso.
- Controlar la transición de las preguntas propias del autor desde BORRADOR hacia PENDIENTE_REVISION.
- Facilitar filtros, consulta de detalles y paginación para las preguntas del autor.
- Permitir registrar revisores y asignarlos sin duplicaciones a preguntas pendientes.
- Implementar notificación por correo, estados persistentes y reintentos de fallos.
- Documentar el diseño y validar reglas de negocio mediante pruebas unitarias automatizadas.

1.4.- Alcance y exclusiones

Incluido en la primera iteración	Fuera del alcance actual
HU-01 a HU-04, formularios Swing, filtros, SQLite y correo SMTP con reintentos	Calificación de preguntas por revisores, aprobación/rechazo y ciclo editorial completo.
Roles demostrativos AUTOR y ADMINISTRADOR	Autenticación real, gestión de contraseñas y permisos de usuarios productivos.
Base de datos local por instalación	Servidor compartido, sincronización entre computadores o servicio web.
Pruebas JUnit y operación local desde Java 21 + Maven	Despliegue empresarial, alta disponibilidad o SLA de producción.

Tabla 1.1. Alcance de la primera iteración

2.- Actores y términos

Actor / componente	Responsabilidad	Acceso actual
Autor	Registrar preguntas, enviar borradores propios a revisión, consultar preguntas propias.	Perfil demostrativo AUTOR_DEMO.
Administrador	Registrar revisores, listar pendientes, asignar docentes, revisar asignaciones y reintentar notificaciones.	Perfil demostrativo ADMIN_DEMO.
Docente revisor	Recibir el correo de asignación.	No existe una interfaz ni login de revisor en esta iteración.
Servidor SMTP	Autenticar el remitente y aceptar o rechazar el transporte del correo.	Dependencia externa opcional para la asignación, necesaria para envío real.
SQLite	Persistir datos en archivo local.	Acceso JDBC desde repositorios de infraestructura.

Tabla 2.1. Actores y responsabilidades

2.1.- Glosario

Término	Significado
BORRADOR	Estado inicial de una pregunta registrada.
PENDIENTE_REVISION	Pregunta enviada por su autor para que pueda asignarse un docente revisor.
Asignación	Relación única entre una pregunta y un docente revisor, con administrador y fecha.
Notificación pendiente	Registro de una asignación cuyo email todavía no figura como enviado.
STARTTLS	Negociación obligatoria de cifrado antes de autenticar y enviar por SMTP.
MVC	Modelo-Vista-Controlador: separación entre presentación, interacción y negocio.
C4	Modelo de arquitectura: contexto, contenedores, componentes y código.

Tabla 2.2. Glosario de términos

2.2.- Reglas de negocio transversales

- RN-01: cada nueva pregunta inicia en el estado BORRADOR y recibe un identificador único.
- RN-02: solo un autor puede enviar a revisión una pregunta propia que esté en BORRADOR.
- RN-03: la consulta «Mis preguntas» únicamente recupera preguntas del autor demostrativo en sesión.
- RN-04: únicamente el rol ADMINISTRADOR puede registrar y asignar revisores.
- RN-05: solo se permite asignar un revisor a una pregunta en estado PENDIENTE_REVISION.
- RN-06: una pareja (pregunta_id, revisor_id) no puede repetirse; sí son posibles varios revisores distintos para una pregunta.
- RN-07: al guardar una asignación nueva se registra una notificación PENDIENTE; si el SMTP acepta el mensaje y se actualiza SQLite, pasa a ENVIADA.
- RN-08: un fallo de correo no revierte la asignación ya persistida; el administrador puede reintentar.
- RN-09: el correo se dirige a la dirección del docente revisor, no necesariamente al remitente SMTP.

3.- Especificación de requisitos

3.1.- Requisitos funcionales (RF)

ID	Requisito funcional	HU
RF-01	Registrar pregunta de selección múltiple	HU-01
RF-02	Validar contexto, enunciado, justificación, bibliografía, competencia, tema, subtema, dificultad y respuesta	HU-01
RF-03	Exigir exactamente cuatro distractores válidos y sin duplicidad; respuesta diferente	HU-01
RF-04	Persistir pregunta nueva como BORRADOR en SQLite	HU-01
RF-05	Consultar por identificador una pregunta del autor	HU-02
RF-06	Cambiar BORRADOR a PENDIENTE_REVISION solo para su propietario	HU-02
RF-07	Mostrar estado de pregunta con diferenciación visual	HU-02 / 03
RF-08	Listar solamente preguntas propias del autor	HU-03
RF-09	Filtrar por estado y dificultad	HU-03
RF-10	Paginar resultados y mostrar total	HU-03

ID	Requisito funcional	HU
RF-11	Visualizar información detallada de una pregunta	HU-03
RF-12	Registrar docente revisor con identificador, nombre y correo	HU-04
RF-13	Rechazar revisores con ID o correo duplicado	HU-04
RF-14	Listar preguntas con PENDIENTE_REVISION	HU-04
RF-15	Asignar uno o más revisores a una pregunta pendiente	HU-04
RF-16	Rechazar duplicidad de la pareja pregunta-revisor	HU-04
RF-17	Consultar docentes asignados a una pregunta	HU-04
RF-18	Enviar correo tras nueva asignación con datos de pregunta	HU-04
RF-19	Registrar PENDIENTE/ENVIADA e intentos en SQLite	HU-04
RF-20	Conservar asignación al fallar SMTP y permitir reintentos	HU-04
RF-21	Permitir perfiles demostrativos Autor y Administrador	Transversal
RF-22	Crear automáticamente base SQLite local en primera ejecución	Transversal

Tabla 3.1. Catálogo de requisitos funcionales

3.2.- Requisitos no funcionales (RNF)

ID	Atributo	Requisito / meta	Medición / verificación	Situación
RNF-01	Modificabilidad	Añadir otro proveedor de notificación mediante INotificadorRevision, sin cambiar lógica de asignación.	Revisión de clases impactadas; prueba de sustitución.	Arquitectura existente; meta propuesta.

ID	Atributo	Requisito / meta	Medición / verificación	Situación
RNF-02	Mantenibilidad	Preservar capas presentación, dominio e infraestructura y contratos de repositorios.	Inspección de dependencias, revisión de PR.	Estructura verificada.
RNF-03	Portabilidad	Ejecutar con JDK 21 y Maven en un equipo con entorno gráfico.	Compilar y abrir desde copia limpia en otro PC.	Pendiente de validar en segundo equipo.
RNF-04	Integridad	Impedir duplicados de asignación y mantener claves/relaciones en SQLite.	Pruebas de restricciones y transacciones.	Implementado y probado.
RNF-05	Seguridad SMTP	Exigir STARTTLS y no incluir credenciales en GitHub.	Inspección de código, .gitignore, variables de entorno.	Implementado.
RNF-06	Capacidad de recuperación	Conservar asignación y reintentar email cuando falla SMTP.	Pruebas de fallo/reintento y persistencia.	Implementado y probado.
RNF-07	Trazabilidad	Registrar fecha, administrador, revisor, pregunta e intentos de email.	Inspeccionar tablas en SQLite.	Implementado.
RNF-08	Usabilidad	Completar registro/asignación sin ayuda adicional tras conocer el flujo.	≥ 80 % de tareas completadas por participantes.	Meta PROPUESTA; no medida.
RNF-09	Respuesta interfaz	Evitar bloqueo perceptible en consultas y envío.	Evaluación manual de interacción; meta ≤ 3 s para consulta de 100 preguntas locales.	Uso SwingWorker observado; medición pendiente.
RNF-10	Pruebas	Ejecutar pruebas JUnit 5 en construcción local reproducible.	mvn clean test finaliza BUILD SUCCESS.	180 éxitos comunicados; repetir en copia limpia.
RNF-11	Instalación	Crear automáticamente archivo data/banco_preguntas.db.	Arranque sobre carpeta nueva.	Implementación verificada.
RNF-12	Confidencialidad	Evitar registrar contraseñas SMTP en archivos o logs de negocio.	Revisión de credenciales y errores almacenados.	No se guarda contraseña; revisar logs de entorno.

ID	Atributo	Requisito / meta	Medición / verificación	Situación
RNF-13	Evolutividad	Permitir nuevos validadores sin modificar el servicio central.	Inyección de IValidadorPregunta adicional.	Diseño compatible; escenario propuesto.
RNF-14	Compatibilidad	Aislar comunicación SMTP de las reglas de asignación.	Inyectar doble de prueba y proveedor real.	Implementado.
RNF-15	Disponibilidad	Sin conexión SMTP debe conservarse el funcionamiento local de asignación.	Desconectar SMTP y verificar operación.	Comportamiento verificado manualmente.

Tabla 3.2. Matriz de requisitos no funcionales

3.3.- Restricciones y supuestos

- Restricción académica: implementación Java Desktop, monolito de tres capas, patrón MVC, principios SOLID y patrones GoF.
- Se asume acceso local a un equipo con interfaz gráfica; Google Cloud Shell exige un servidor gráfico virtual, mientras que Windows con escritorio no.
- La sesión de usuario es demostrativa. Los controles de rol en servicio no sustituyen autenticación productiva.
- Las credenciales del proveedor SMTP pertenecen al usuario que ejecuta la aplicación y no están distribuidas en el repositorio.
- El esquema SQLite local no constituye base de datos multiusuario compartida ni garantiza copias de respaldo automáticas.

4.- Historias de usuario y especificaciones

A continuación se documentan las cuatro historias con formato «Como [actor], quiero [objetivo], para [beneficio]», condiciones, flujos, excepciones y criterios expresados como Dado/Cuando/Entonces. Los identificadores HU y RF permiten comprobar trazabilidad entre requisitos, casos de uso y pruebas.

4.1.- HU-01 · Registrar pregunta Saber Pro

Campo	Descripción
ID / prioridad	HU-01 · Alta · Primera iteración
Actor principal	Autor
Historia de usuario	Como autor de preguntas, quiero registrar preguntas de selección múltiple con única respuesta siguiendo campos de diseño centrado en evidencia, para alimentar el Banco de Preguntas Saber Pro.

Campo	Descripción
Objetivo / valor	Obtener preguntas completas, válidas y persistidas para revisión futura.
Precondiciones	Aplicación iniciada; Perfil demostrativo Autor seleccionado; Usuario autor_id definido en la sesión
Poscondiciones	Pregunta nueva persistida con identificador UUID; Estado inicial BORRADOR; La pregunta aparece en la consulta de HU-03
Reglas de negocio	Exactamente cuatro distractores; Respuesta correcta independiente de los distractores; Registro único con estado BORRADOR

Tabla 4.1. Ficha de HU-01

4.1.1.- Flujo principal del caso de uso

1. El autor selecciona el perfil Autor y accede al formulario de registro.
2. Completa contexto, pregunta directa, cuatro distractores y respuesta correcta.
3. Agrega justificación, bibliografía, competencia, tema, subtema y dificultad.
4. Presiona la acción de guardar.
5. El controlador entrega la solicitud al servicio de registro.
6. Los validadores comprueban obligatoriedad, cantidad de distractores y opciones distintas.
7. El repositorio SQLite registra pregunta y distractores dentro de una transacción.
8. La interfaz confirma el resultado y permite realizar una nueva operación.

4.1.2.- Flujos alternativos y excepciones

- A1: Campo obligatorio vacío: no se persiste y se informa validación.
- A2: Distractor repetido o distinto de la cantidad requerida: se rechaza el registro.
- A3: Respuesta correcta coincide con un distractor: se reporta error.
- A4: Fallo SQLite: se informa error; no debe considerarse guardada una pregunta parcial.

4.1.3.- Criterios de aceptación verificables

CA-01-01. Dado el usuario dispone de un formulario válido, cuando guarda la pregunta, entonces se crea una pregunta BORRADOR con un identificador único.

CA-01-02. Dado falta un campo obligatorio, cuando intenta registrar, entonces el sistema informa el campo y evita persistir.

CA-01-03. Dado existen menos o más de cuatro distractores, cuando intenta registrar, entonces se rechaza la estructura.

CA-01-04. Dado dos distractores tienen el mismo contenido normalizado, cuando intenta registrar, entonces se reporta duplicidad.

CA-01-05. Dado la respuesta coincide con un distractor, cuando intenta registrar, entonces se rechaza la combinación inválida.

CA-01-06. Dado el registro fue exitoso, cuando abre Mis preguntas, entonces la pregunta aparece entre las de su autor.

4.1.4.- Componentes y evidencia de verificación

Clases vinculadas en el repositorio: Pregunta, PreguntaServiceImpl, ServicioRegistroAutorizado, ValidadorCamposObligatorios, ValidadorOpciones, SqlitePreguntaRepository, RegistroPreguntaController, RegistroPreguntaView.

Pruebas vinculadas: PreguntaTest, PreguntaServiceImplTest, ValidadorCamposObligatoriosTest, ValidadorOpcionesTest, SqlitePreguntaRepositoryTest.

4.1.5.- Definition of Ready (DoR) — verificación previa al desarrollo

Las condiciones siguientes deben verificarse al iniciar la historia. Su cumplimiento histórico requiere evidencia del tablero y del equipo; no se presume automáticamente.

☐ Alcance y campos obligatorios definidos conforme a HU-01.

☐ Reglas de validación de opciones y respuesta correcta aclaradas.

☐ Actor Autor y condiciones de acceso identificados.

☐ Prototipo de formulario disponible para discusión.

☐ Criterios de aceptación verificables y casos de prueba definidos.

☐ Dependencias del servicio y repositorio identificadas.

☐ Historia estimada y asignada en el Sprint por el equipo.

4.1.6.- Definition of Done (DoD) — condición de cierre

Los elementos técnicos se contrastan con el código y pruebas; los artefactos de gestión y aceptación requieren sus respectivas evidencias.

☐ Modelo Pregunta y estado BORRADOR integrados.

☐ Vista Swing y controlador MVC operativos.

☐ Servicio de registro y validadores estructurales implementados.

☐ Persistencia de preguntas y distractores en SQLite verificada.

☐ Pruebas de campos, opciones y registro ejecutadas.

☐ Cambios revisados y fusionados a main mediante Pull Request.

☐ Evidencias de criterios de aceptación vinculadas a Jira.

4.2.- HU-02 · Enviar borrador a revisión

Campo	Descripción
ID / prioridad	HU-02 · Alta · Primera iteración
Actor principal	Autor
Historia de usuario	Como autor de preguntas, quiero cambiar el estado de mis preguntas de BORRADOR a PENDIENTE_REVISION, para permitir que el administrador asigne revisores.
Objetivo / valor	Autorizar revisión académica de contenido ya registrado.
Precondiciones	Existe pregunta registrada por el autor actual; Su estado es BORRADOR
Poscondiciones	Estado persistido en PENDIENTE_REVISION; Pregunta visible al administrador para asignación
Reglas de negocio	Solo el dueño en sesión modifica su estado; Solo BORRADOR -> PENDIENTE_REVISION; La transición queda almacenada en SQLite

Tabla 4.2. Ficha de HU-02

4.2.1.- Flujo principal del caso de uso

1. El autor abre «Enviar a revisión (HU-02)».
2. Busca la pregunta utilizando su identificador.
3. El servicio verifica que exista y que pertenezca al autor de la sesión.
4. Comprueba que su estado actual sea BORRADOR.
5. El autor solicita el cambio.
6. El repositorio actualiza de manera condicionada el estado en SQLite.
7. La interfaz muestra la confirmación y el nuevo estado.

4.2.2.- Flujos alternativos y excepciones

- A1: No existe pregunta: se informa que no fue encontrada.
- A2: Pregunta de otro autor: se deniega el cambio.
- A3: Estado distinto de BORRADOR: se rechaza el reenvío.
- A4: Actualización concurrente o fallida: se informa que no se actualizó.

4.2.3.- Criterios de aceptación verificables

CA-02-01. Dado una pregunta propia está en BORRADOR, cuando su autor la envía a revisión, entonces queda en PENDIENTE_REVISION.

CA-02-02. Dado la pregunta pertenece a otro autor, cuando se solicita el cambio, entonces se deniega la transición.

CA-02-03. Dado la pregunta ya está en PENDIENTE_REVISION, cuando se vuelve a enviar, entonces no se ejecuta transición duplicada.

CA-02-04. Dado el identificador es inexistente, cuando el autor realiza la consulta, entonces el sistema informa que no existe.

4.2.4.- Componentes y evidencia de verificación

Clases vinculadas en el repositorio: Pregunta, EstadoPregunta, PreguntaEstadoServiceImpl, SqlitePreguntaRepository, PreguntaEstadoController, PreguntaEstadoView.

Pruebas vinculadas: PreguntaCambioEstadoTest, PreguntaEstadoServiceImplTest, SqliteEstadoPreguntaRepositoryTest.

4.2.5.- Definition of Ready (DoR) — verificación previa al desarrollo

Las condiciones siguientes deben verificarse al iniciar la historia. Su cumplimiento histórico requiere evidencia del tablero y del equipo; no se presume automáticamente.

☐ Estados BORRADOR y PENDIENTE_REVISION acordados.

☐ Transiciones válidas e inválidas especificadas.

☐ Reglas de propiedad de la pregunta y rol Autor definidas.

☐ HU-01 disponible para registrar los datos de prueba.

☐ Prototipo y escenarios de error identificados.

☐ Criterios de aceptación acordados y estimación del Sprint registrada.

4.2.6.- Definition of Done (DoD) — condición de cierre

Los elementos técnicos se contrastan con el código y pruebas; los artefactos de gestión y aceptación requieren sus respectivas evidencias.

☐ Cambio de estado implementado en servicio y repositorio.

☐ Control de rol Autor, propiedad y estado BORRADOR probado.

☐ Vista Swing con estado y controles de revisión integrada.

☐ Rechazo de envíos repetidos y preguntas ajenas validado.

☐ Pruebas de estado y persistencia ejecutadas.

☐ Integración a main y evidencias del Sprint documentadas.

4.3.- HU-03 · Consultar y filtrar mis preguntas

Campo	Descripción
ID / prioridad	HU-03 · Alta · Primera iteración
Actor principal	Autor
Historia de usuario	Como autor de preguntas, quiero listar las preguntas que he creado con filtros y paginación, para encontrarlas y revisar su información.

Campo	Descripción
Objetivo / valor	Facilitar búsqueda, seguimiento y visualización de preguntas propias.
Precondiciones	Existe perfil Autor seleccionado; Base de datos disponible; puede estar vacía
Poscondiciones	Listado de preguntas propias con total y página; Estado de los registros sin modificaciones
Reglas de negocio	Lista limitada al autor en sesión; Filtro por estado y dificultad; Tamaño de página de la UI: 10 registros; Paginación controlada sin alterar datos

Tabla 4.3. Ficha de HU-03

4.3.1.- Flujo principal del caso de uso

1. El autor abre «Mis preguntas (HU-03)».
2. El sistema carga la primera página de preguntas de ese autor.
3. El autor selecciona opcionalmente un estado y nivel de dificultad.
4. Solicita aplicar filtros.
5. El controlador consulta el servicio y el repositorio por autor, filtros, límite y desplazamiento.
6. La interfaz muestra filas y total de resultados.
7. El autor navega Anterior/Siguiente y puede abrir «Ver detalles».

4.3.2.- Flujos alternativos y excepciones

- A1: Sin resultados: se presenta una lista vacía y el estado correspondiente.
- A2: Número de página inválido: el servicio rechaza parámetros fuera del rango permitido.
- A3: Usuario no AUTOR: se rechaza la consulta.
- A4: Fallo SQLite: se notifica que no fue posible completar la consulta.

4.3.3.- Criterios de aceptación verificables

CA-03-01. Dado un autor tiene preguntas registradas, cuando abre Mis preguntas, entonces ve solo sus preguntas.

CA-03-02. Dado existen preguntas en más de una página, cuando selecciona Siguiente, entonces se muestran elementos del siguiente bloque.

CA-03-03. Dado el autor selecciona estado PENDIENTE_REVISION, cuando aplica filtro, entonces la tabla muestra únicamente coincidencias.

CA-03-04. Dado elige un nivel de dificultad, cuando aplica filtro, entonces se muestran solo registros coincidentes.

CA-03-05. Dado existe una pregunta en la tabla, cuando selecciona Ver detalles, entonces se muestran sus campos de consulta.

4.3.4.- Componentes y evidencia de verificación

Clases vinculadas en el repositorio: ConsultaPreguntaServiceImpl, PaginaPreguntas, SqlitePreguntaRepository, ConsultaPreguntaController, ConsultaPreguntaView.

Pruebas vinculadas: ConsultaPreguntaServiceImplTest, SqliteConsultaPreguntaRepositoryTest, ConsultaPreguntaControllerTest.

4.3.5.- Definition of Ready (DoR) — verificación previa al desarrollo

Las condiciones siguientes deben verificarse al iniciar la historia. Su cumplimiento histórico requiere evidencia del tablero y del equipo; no se presume automáticamente.

☐ Necesidad de consulta por autor y campos del listado definidos.

☐ Filtros por estado y dificultad especificados.

☐ Tamaño y navegación de paginación decididos.

☐ Política de consulta de preguntas propias definida.

☐ Prototipo de consulta y casos de prueba preparados.

☐ Historia valorada y asignada en el tablero Sprint.

4.3.6.- Definition of Done (DoD) — condición de cierre

Los elementos técnicos se contrastan con el código y pruebas; los artefactos de gestión y aceptación requieren sus respectivas evidencias.

☐ Listado SQLite limitado al autor en sesión implementado.

☐ Filtros, conteo, paginación y consulta de detalles operativos.

☐ Ventana Swing y controlador MVC integrados.

☐ Casos sin resultados y navegación comprobados.

☐ Pruebas del servicio, consulta y controlador ejecutadas.

☐ Pull Request fusionado y evidencia de criterios archivada.

4.4.- HU-04 · Asignar y notificar revisores

Campo	Descripción
ID / prioridad	HU-04 · Alta · Primera iteración
Actor principal	Administrador
Historia de usuario	Como administrador, quiero asignar al menos un docente revisor a las preguntas en estado PENDIENTE_REVISION, para que otros docentes puedan revisarlas y recibir un correo de notificación.
Objetivo / valor	Formalizar asignaciones trazables, impedir duplicados y avisar al docente.

Campo	Descripción
Precondiciones	Perfil ADMINISTRADOR; Existe una pregunta PENDIENTE_REVISION; Existe un docente revisor válido registrado
Poscondiciones	Asignación persistida; Notificación ENVIADA o PENDIENTE según resultado real; Datos disponibles en Ver asignados
Reglas de negocio	Pregunta en PENDIENTE_REVISION; Administrador obligatorio; Pareja pregunta-revisor única; Al menos un revisor puede asignarse; Notificaciones por SMTP y reintentos

Tabla 4.4. Ficha de HU-04

4.4.1.- Flujo principal del caso de uso

1. El administrador abre el perfil HU-04 y consulta preguntas pendientes.
2. Si hace falta un docente, abre Gestión de revisores y lo registra.
3. Selecciona una pregunta pendiente y un revisor existente.
4. Pulsa «Asignar revisor».
5. El servicio verifica permisos, pregunta, estado y revisor.
6. SQLite inserta una asignación única y crea notificación PENDIENTE mediante trigger.
7. Se intenta enviar correo SMTP al destinatario registrado.
8. Si SMTP acepta el mensaje y el registro persiste, se marca ENVIADA; si falla, queda PENDIENTE.
9. El administrador puede consultar Ver asignados o Reintentar notificaciones.

4.4.2.- Flujos alternativos y excepciones

- A1: No ADMINISTRADOR: error de autorización.
- A2: La pregunta no está pendiente: no se asigna.
- A3: Revisor no registrado: no se asigna.
- A4: Misma pareja pregunta/revisor: no se crea segunda asignación.
- A5: SMTP no configurado o rechaza: se guarda la asignación y el estado PENDIENTE.
- A6: Reintento correcto: el correo es aceptado y el estado pasa a ENVIADA.

4.4.3.- Criterios de aceptación verificables

CA-04-01. Dado una pregunta está pendiente y el revisor existe, cuando el administrador asigna, entonces se guarda una nueva relación única.

CA-04-02. Dado el revisor ya fue asignado a esa pregunta, cuando se intenta repetir, entonces no se duplica ni se genera otro envío por esa operación.

CA-04-03. Dado SMTP está correctamente configurado, cuando se registra una asignación nueva, entonces se intenta enviar el correo al revisor.

CA-04-04. Dado SMTP no está disponible, cuando se guarda una nueva asignación, entonces la asignación persiste y la notificación queda PENDIENTE.

CA-04-05. Dado una notificación está PENDIENTE y SMTP vuelve a funcionar, cuando el administrador pulsa Reintentar notificaciones, entonces se registra la aceptación como ENVIADA.

CA-04-06. Dado un usuario distinto de administrador solicita asignar, cuando se procesa la solicitud, entonces la operación se deniega.

CA-04-07. Dado el identificador o correo del revisor ya existe, cuando se intenta registrar, entonces el sistema rechaza el duplicado.

4.4.4.- Componentes y evidencia de verificación

Clases vinculadas en el repositorio: AsignacionRevisionServiceImpl, RegistroRevisorServiceImpl, ReintentoNotificacionesServiceImpl, SqliteAsignacionRevisionRepository, SqliteNotificacionRevisionRepository, NotificadorRevisionEmail, NotificadorRevisionConRegistro, AsignacionRevisionController, AsignacionRevisionView, RegistroRevisorView.

Pruebas vinculadas: AsignacionRevisionServiceImplTest, AsignacionRevisionNotificacionTest, RegistroRevisorServiceImplTest, ReintentoNotificacionesServiceImplTest, NotificadorRevisionConRegistroTest, NotificadorRevisionEmailTest, SqliteAsignacionRevisionRepositoryTest, RegistroRevisorControllerTest.

4.4.5.- Definition of Ready (DoR) — verificación previa al desarrollo

Las condiciones siguientes deben verificarse al iniciar la historia. Su cumplimiento histórico requiere evidencia del tablero y del equipo; no se presume automáticamente.

☐ Reglas para preguntas pendientes y asignaciones definidas.

☐ Datos y validaciones para registrar revisores establecidos.

☐ Política para impedir duplicados definida.

☐ Servicio SMTP y comportamiento ante fallos acordados.

☐ HU-02 disponible con preguntas pendientes para pruebas.

☐ Criterios de aceptación y escenarios de notificación definidos.

☐ Historia estimada y asignada en el Sprint.

4.4.6.- Definition of Done (DoD) — condición de cierre

Los elementos técnicos se contrastan con el código y pruebas; los artefactos de gestión y aceptación requieren sus respectivas evidencias.

☐ Modelos Revisor y AsignacionRevision implementados.

☐ Registro de docentes y restricciones de duplicados probados.

☐ Consulta y asignación de preguntas pendientes funcionales.

☐ Correo SMTP, estados PENDIENTE/ENVIADA y reintentos operativos.

☐ Fallo SMTP sin pérdida de asignación verificado.

☐ Prueba de envío real y recepción confirmada.

☐ Pruebas automatizadas ejecutadas y código fusionado a main.

☐ Credenciales excluidas del repositorio y README documentado.

5.- Modelado de casos de uso UML

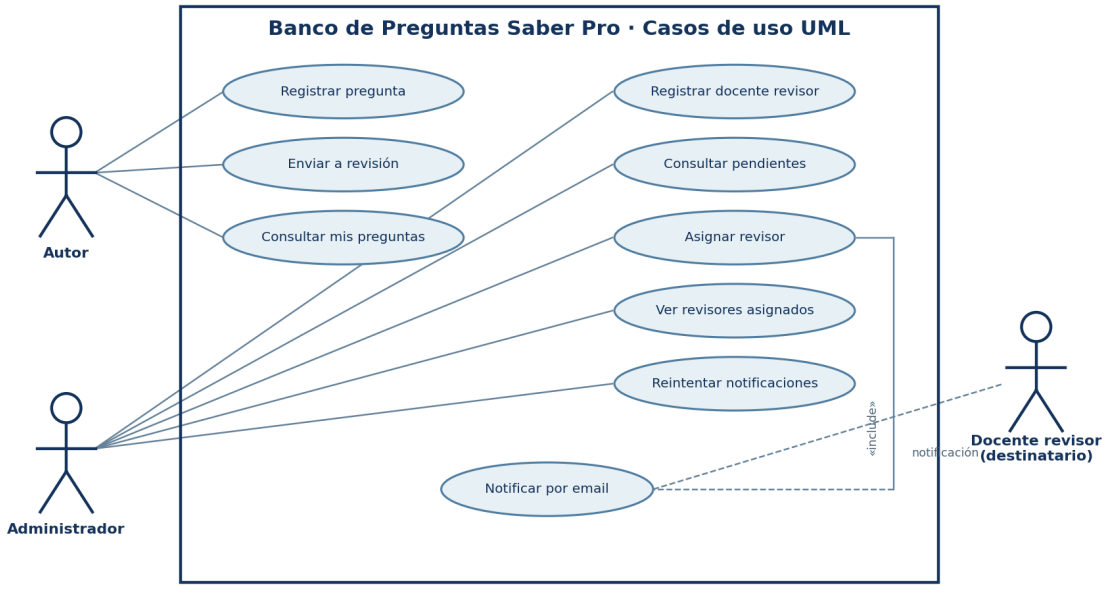


Figura 5.1. Casos de uso vinculados a los perfiles demostrativos. El docente es destinatario de correo, no un usuario autenticado.

5.1.- Catálogo de casos de uso

Caso	Nombre	Actor	Requisitos	Historia
CU-01	Registrar pregunta	Autor	RF-01 a RF-04	HU-01
CU-02	Enviar pregunta a revisión	Autor	RF-05 a RF-07	HU-02
CU-03	Consultar mis preguntas	Autor	RF-08 a RF-11	HU-03
CU-04	Registrar docente revisor	Administrador	RF-12 a RF-13	HU-04
CU-05	Consultar preguntas pendientes	Administrador	RF-14	HU-04
CU-06	Asignar revisor	Administrador	RF-15, RF-16, RF-18, RF-19	HU-04
CU-07	Consultar asignaciones	Administrador	RF-17	HU-04
CU-08	Reintentar notificaciones	Administrador	RF-19, RF-20	HU-04

Tabla 5.1. Catálogo de casos de uso

5.2.- Precondiciones y poscondiciones por caso

CU	Precondición mínima	Poscondición exitosa / alterna
CU-01	Perfil Autor; datos válidos	Pregunta BORRADOR creada / errores no generan registro.
CU-02	Pregunta propia en BORRADOR	Estado PENDIENTE_REVISION / no cambia si hay error.
CU-03	Perfil Autor	Listado filtrado del autor / cero coincidencias posible.
CU-04	Administrador y datos de revisor válidos	Revisor nuevo persistido / duplicado rechazado.
CU-05	Administrador	Lista paginada de preguntas pendientes.
CU-06	Administrador, pregunta pendiente, revisor registrado	Asignación única guardada y correo intentado / pendiente ante fallo.
CU-07	Administrador, pregunta seleccionada	Listado de asignados consultable.
CU-08	Administrador y pendientes registradas	Resultados de reintento y estados actualizados si procede.

Tabla 5.2. Precondiciones y poscondiciones

5.3.- Diagrama de actividad y secuencia

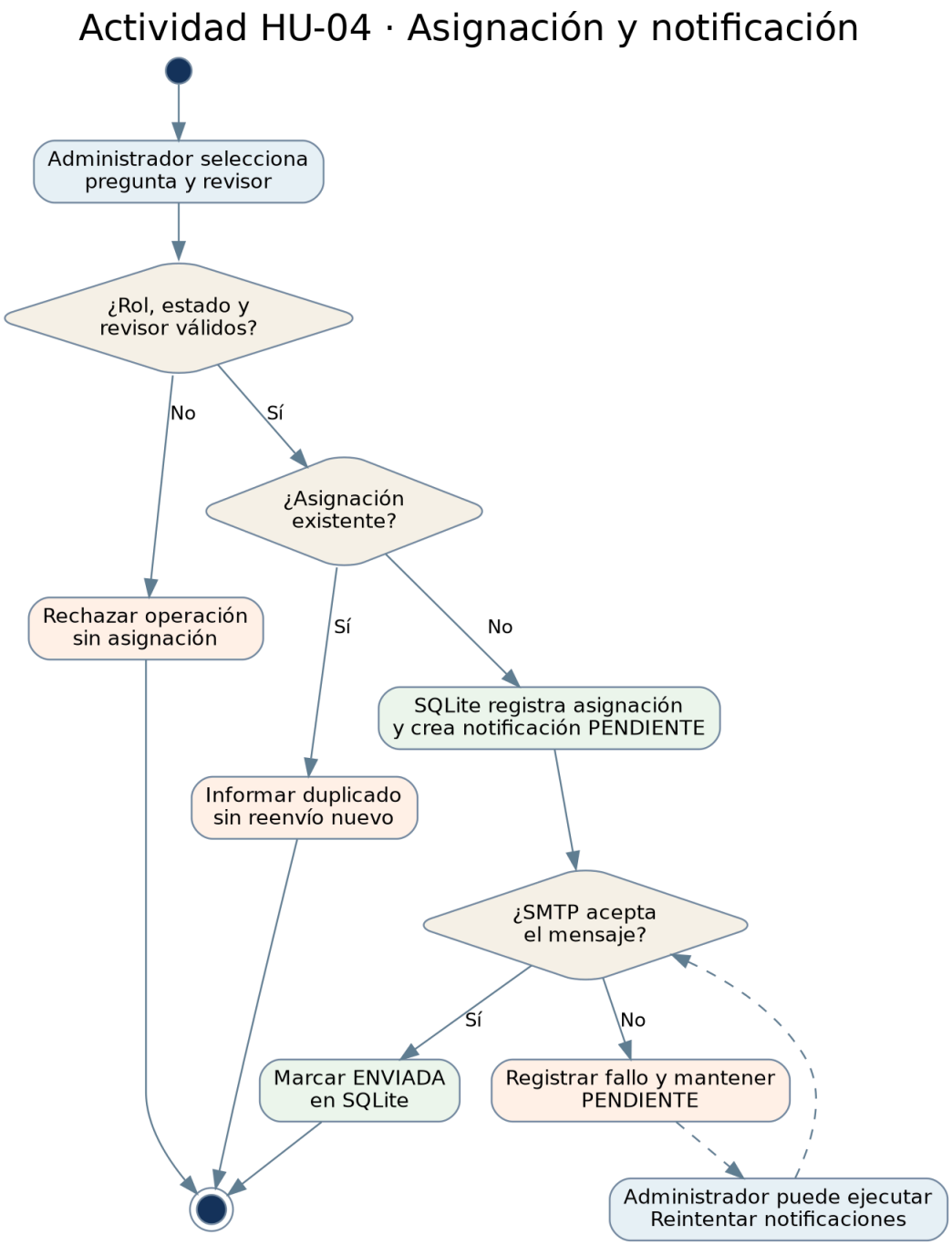


Figura 5.2. Flujo de actividad de una nueva asignación y su notificación (las ramas no exitosas conservan la integridad).

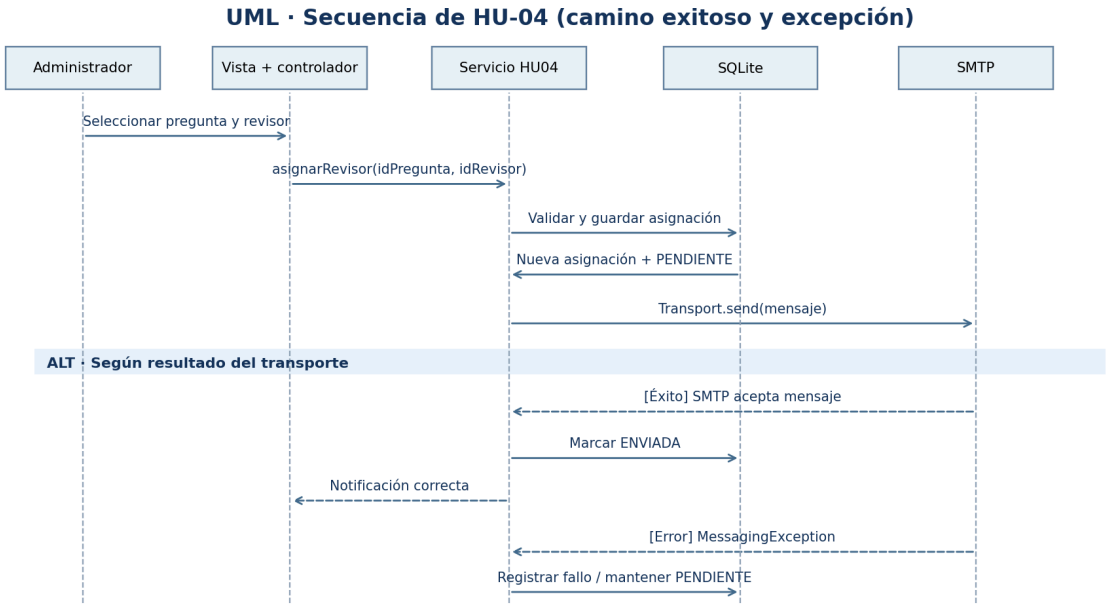


Figura 5.3. Secuencia simplificada de persistencia y notificación. En los fallos, el decorador registra error seguro.

6.- Arquitectura y diseño con modelo C4

6.1.- Principios y decisiones arquitectónicas

El Banco de Preguntas Saber Pro está construido como una arquitectura monolítica modular de escritorio: toda la aplicación se ejecuta dentro de una misma JVM de Java 21. El sistema se organiza en tres capas lógicas (presentación, dominio e infraestructura) y utiliza MVC para separar las vistas Swing de la coordinación de operaciones y de las reglas del negocio. No se implementan microservicios, API REST, servidor web ni autenticación real.

Capa de presentación: las vistas RegistroPreguntaView, PreguntaEstadoView, ConsultaPreguntaView, AsignacionRevisionView y RegistroRevisorView muestran formularios, tablas, diálogos y resultados. Los controladores MVC delegan la operación en servicios; las vistas no deben contener sentencias SQL ni credenciales de correo.

Capa de dominio: Pregunta, Revisor, AsignacionRevision y EstadoPregunta describen entidades, estados y restricciones. Las interfaces I*Service, I*Repository, IValidadorPregunta e INotificadorRevision representan contratos. Los servicios coordinan los casos de uso y validan autorización por perfil demostrativo, pertenencia de preguntas, estado de revisión y duplicados.

Capa de infraestructura: los repositorios Sqlite*Repository implementan persistencia mediante JDBC en data/banco_preguntas.db. NotificadorRevisionEmail usa Jakarta Mail y SMTP/STARTTLS; NotificadorRevisionConRegistro incorpora el seguimiento de estados para los reintentos. En Main se ensamblan las dependencias: los servicios conocen interfaces en vez de implementaciones concretas (inversión de dependencias). Esta separación facilita reemplazar almacenamiento o correo sin trasladar su lógica a las vistas.

Alcance del diseño: se aplican separación de responsabilidades, inyección por constructor y patrones MVC, Repository y Decorator. Estos elementos favorecen la modificabilidad, pero no implican por sí solos que se haya implementado íntegramente Domain-Driven Design (DDD). El acceso por perfiles es demostrativo y la base SQLite es local por instalación.

Decisión	Motivo	Consecuencia
Java Swing y MVC	Cumplimiento de requisito de escritorio; separación de interacción y negocio.	Requiere entorno gráfico y pruebas UI específicas.
SQLite local	Persistencia embebida simple y transacciones; sin servidor.	Cada instalación tiene datos propios; sin sincronización.
Servicios + interfaces	Acoplamiento reducido e inyección de dependencias por constructor.	Permite pruebas con dobles y cambios de persistencia.
SMTP STARTTLS	Notificación solicitada por HU-04, cifrada y autenticada.	Depende de conectividad, proveedor y credenciales externas.
Notificación pendiente persistida	No perder asignación ante fallo del proveedor de correo.	Se implementa reintento; entrega exactamente una vez no garantizada.

Decisión	Motivo	Consecuencia
SwingWorker en vistas	Mover consultas y tareas fuera del hilo gráfico.	Se mantiene interfaz reactiva sin prometer métricas sin medir.

Tabla 6.1. Decisiones arquitectónicas

6.2.- C4 nivel 1: contexto

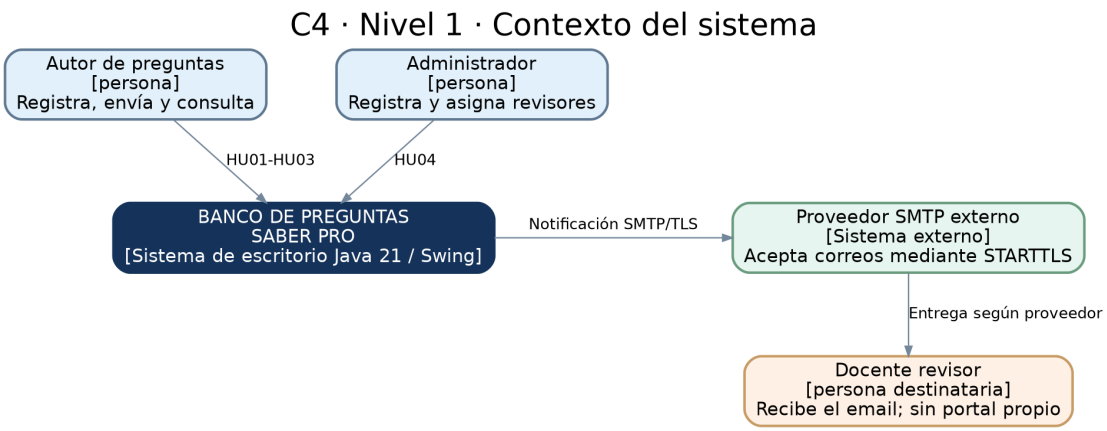


Figura 6.1. Contexto: actores directos, destinatario de notificación y servicio SMTP externo.

6.3.- C4 nivel 2: contenedores

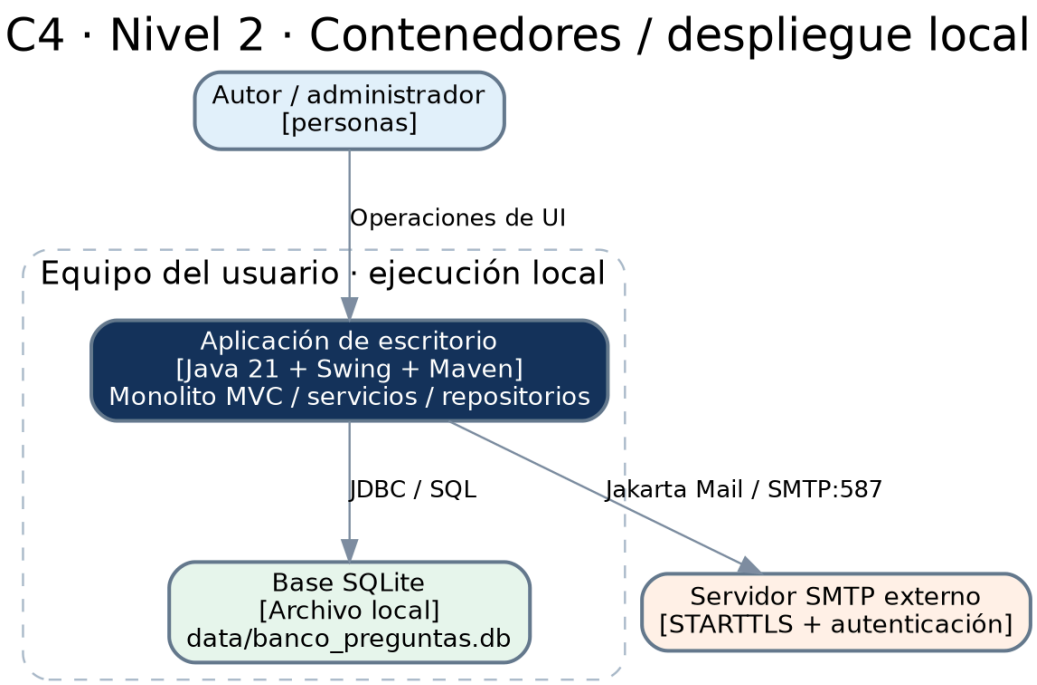


Figura 6.2. Despliegue real: una JVM Java Swing y un archivo SQLite en el computador del usuario.

6.4.- C4 nivel 3: componentes

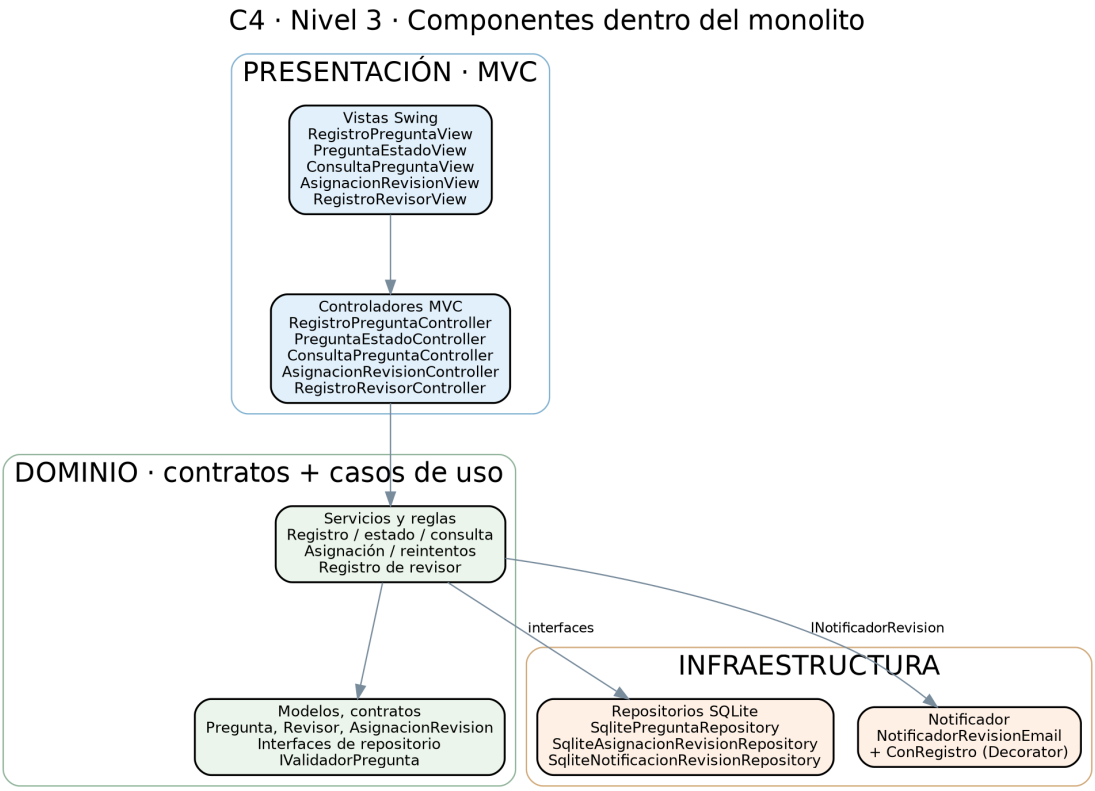


Figura 6.3. Componentes principales de las tres capas dentro del monolito.

6.5.- Mapa de implementación y dependencias

Capa / paquete	Componentes representativos	Responsabilidad
presentation.view	RegistroPreguntaView, PreguntaEstadoView, ConsultaPreguntaView, AsignacionRevisionView, RegistroRevisorView	Formularios, listados, diálogos y avisos.
presentation.controller	*Controller para registrar, cambiar estado, consultar, asignar y registrar revisores	Coordinar entradas de vistas y servicios.
domain.model	Pregunta, EstadoPregunta, Revisor, AsignacionRevision, NotificacionPendiente	Datos y reglas estructurales.
domain.service	*ServiceImpl e interfaces I*Service	Reglas de negocio, roles y orquestación.
domain.repository / validation	Interfaces de repositorio e IValidadorPregunta	Puertos y contratos para la infraestructura.

Capa / paquete	Componentes representativos	Responsabilidad
infrastructure.persistence	Sqlite*Repository	Conexiones JDBC, SQL y transacciones.
infrastructure.notification	NotificadorRevisionEmail, NotificadorRevisionConRegistro	Transporte y registro de resultados SMTP.

Tabla 6.2. Mapa de implementación por capas

7.- Modelo UML, datos y estados

7.1.- Diagrama UML de clases representativas

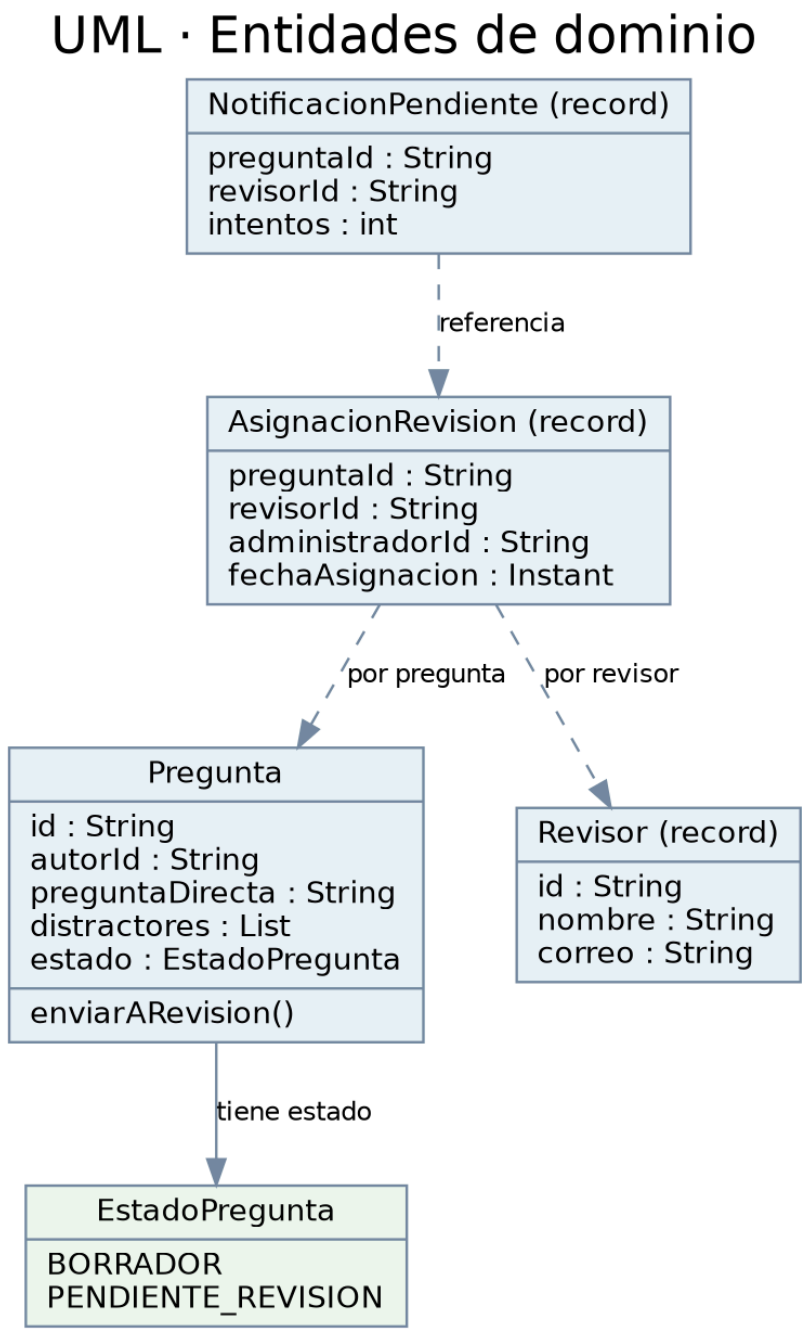


Figura 7.1. Entidades de dominio: pregunta, revisor, asignación y notificaciones.

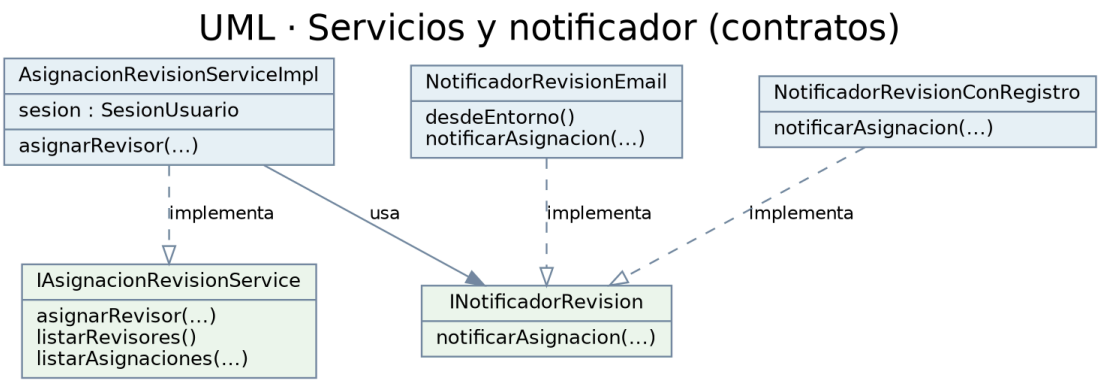


Figura 7.2. Contratos y dependencias de servicios y notificación (vista simplificada).

7.2.- Modelo de datos persistentes

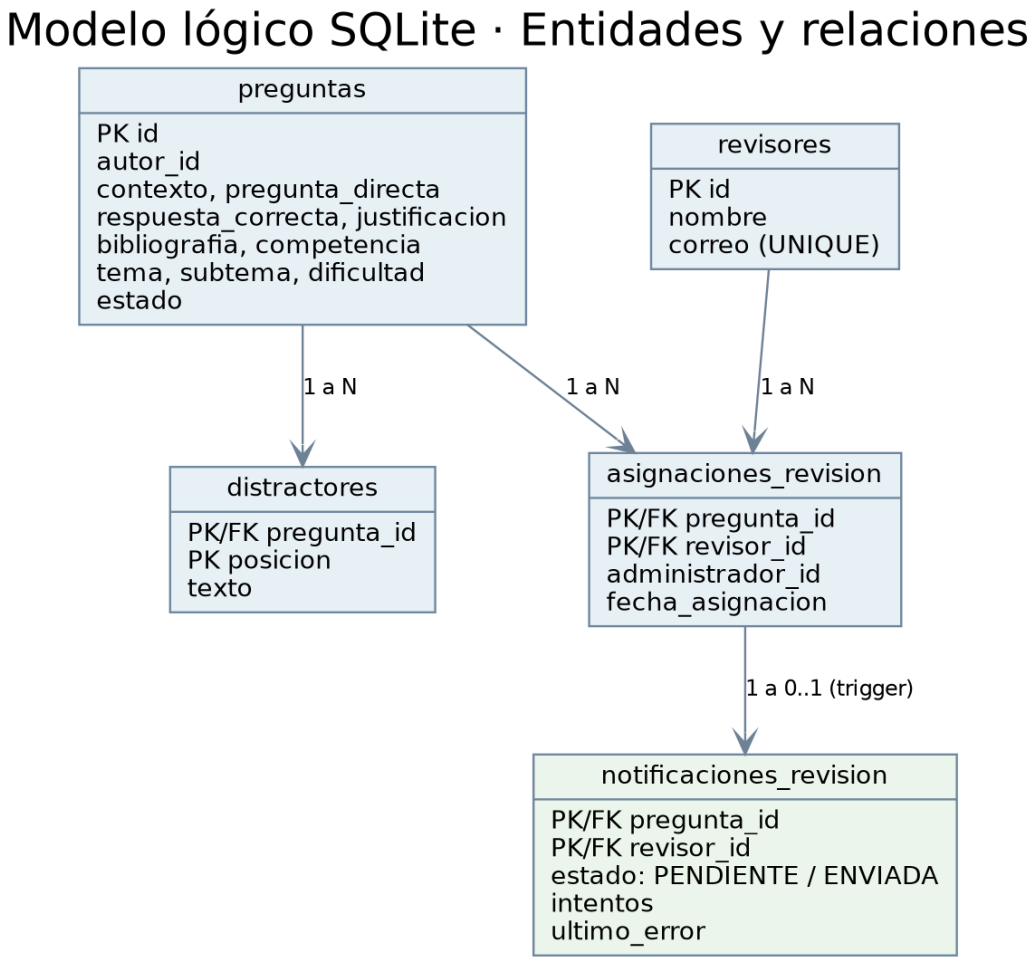


Figura 7.3. Esquema lógico SQLite y claves de relación. Los datos de usuario son IDs demostrativos.

La tabla preguntas almacena campos académicos y estado. distractores relaciona opciones mediante pregunta_id y posición. revisores almacena docentes y correo único sin distinción de mayúsculas.

asignaciones_revision posee clave primaria compuesta (pregunta_id, revisor_id), identificador de administrador y fecha. notificaciones_revision conserva estado, intentos y último error seguro, creando registros PENDIENTE mediante trigger posterior a una asignación nueva.

7.3.- Diagramas de estados

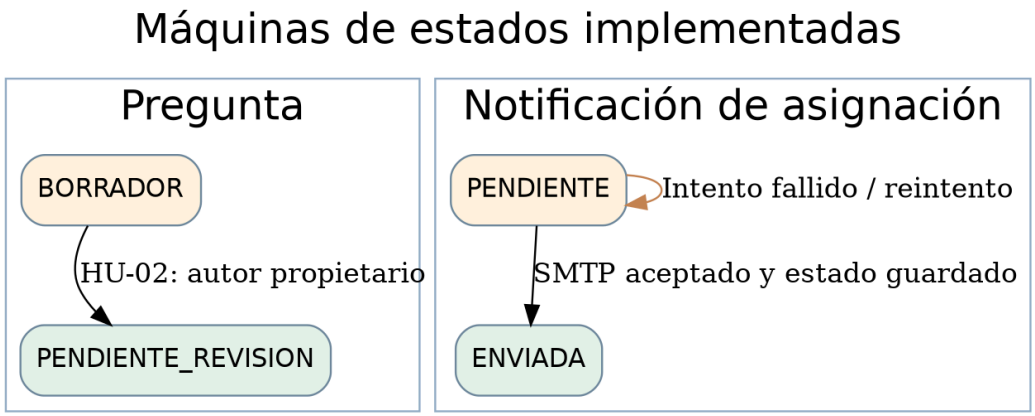


Figura 7.4. Transiciones implementadas: preguntas y notificaciones. No se documentan como existentes estados de evaluación no implementados.

7.4.- Invariantes de persistencia

- Las nuevas preguntas se registran en BORRADOR; el repositorio verifica condiciones de autor y estado al enviar a revisión.
- Las asignaciones se realizan únicamente a preguntas pendientes y no permiten duplicación de pregunta/revisor.
- El trigger que crea una notificación PENDIENTE se ejecuta en SQLite al insertar una nueva asignación.
- El estado ENVIADA se marca solo después del retorno exitoso del notificador, aunque pueden existir duplicados de correo si una aceptación SMTP ocurre y falla el registro local del resultado.
- Una base nueva no contiene los registros creados previamente en otro equipo porque data/ no se comparte por Git.

8.- Patrones de diseño y SOLID

Patrón o principio	Dónde se aplica	Beneficio / límite
MVC	Vistas Swing, controladores y objetos de dominio	Separa eventos de UI de servicios; el modelo no depende de Swing.
Repository	Interfaces I*Repository + implementaciones Sqlite*Repository	Separa SQL y servicios de negocio.

Patrón o principio	Dónde se aplica	Beneficio / límite
Decorator (GoF)	NotificadorRevisionConRegistro implementa INotificadorRevision y envuelve notificador base	Añade trazabilidad sin alterar el componente SMTP.
Strategy / Policy por interfaz	IValidadorPregunta y varias implementaciones concretas	Valida en secuencia lista de reglas inyectadas. Similar a Strategy; no es un motor dinámico de plugins.
Inyección por constructor	Servicios reciben repositorios/notificadores/sesiones	Permite reemplazar dependencias durante pruebas.
SRP	Repositorios SQL, servicios de negocio y notificador SMTP separados	Responsabilidades diferenciadas.
OCP	Añadir otra implementación de IValidadorPregunta o INotificadorRevision	Extender sin reescribir la mayor parte de la regla principal.
LSP / ISP	Implementaciones respetan contratos de sus interfaces	Exige verificar sustitución en pruebas; no se asume prueba formal exhaustiva.
DIP	Servicios dependen de interfaces de repositorio y notificación	Menor acoplamiento con SQLite y Gmail.

Tabla 8.1. Patrones y principios aplicados

8.1.- Ejemplo concreto: decorador de notificación

AsignacionRevisionServiceImpl depende de INotificadorRevision. Main compone NotificadorRevisionEmail con NotificadorRevisionConRegistro y el repositorio de estados. El decorador invoca notificarAsignacion(...); ante resultado aceptado marca ENVIADA; ante RuntimeException registra un fallo genérico y conserva PENDIENTE. Esta solución atiende el requisito de reintento y reduce acoplamiento con el proveedor SMTP.

8.2.- Responsabilidad de la sesión demostrativa

El sistema configura AUTOR_DEMO y ADMIN_DEMO desde el selector inicial. Las comprobaciones de RolUsuario en servicios son útiles para reglas académicas, pero no representan una frontera segura de autenticación real: cualquier persona puede elegir Administrador en la pantalla de inicio. No se debe describir el producto como un sistema de acceso seguro en producción.

9.- Escenarios de calidad y medición

9.1.- Escenario obligatorio de modificabilidad

CONTEXTO	ESTÍMULO	RESPUESTA	MEDICIÓN DE CALIDAD	RESULTADO ESPERADO
Aplicación monolítica en tres capas; HU-04 y pruebas JUnit operativas.	Se requiere sustituir Gmail por otro servidor SMTP autorizado.	Se cambia la configuración o la implementación de INotificadorRevision; no se modifica AsignacionRevisionServiceImpl.	Revisar clases cambiadas; compilar y ejecutar mvn clean test; registrar tiempo real de cambio.	META: cambio localizado en notificación/configuración y cero regresiones. El ensayo cronometrado está pendiente.
Servicio de registro con validadores desacoplados por IValidadorPregunta.	Surge una regla estructural nueva para las preguntas.	Se crea un validador adicional y se inyecta en el registro sin reescribir PreguntaServiceImpl.	Contar archivos del dominio modificados y pruebas nuevas; verificar que la suite permanezca verde.	META: ampliación por nueva clase de validación e integración; medición de esfuerzo pendiente.
Servicios dependen de interfaces de repositorio y la app almacena datos en SQLite.	Se solicita persistir datos mediante otra implementación compatible.	Se implementa el contrato de repositorio y se cambia el ensamblaje en Main.	Comprobar que servicios de negocio no dependan de SQL; ejecutar casos HU-01 a HU-04.	META: conservar las reglas de negocio sin reescritura; la migración no está realizada.

Las metas anteriores son criterios propuestos para un ensayo de modificabilidad; no son mediciones ejecutadas. El resultado debe registrarse después de realizar un cambio controlado, consignando fecha, duración, clases afectadas y evidencia de pruebas.

Tabla 9.1. Escenario de modificabilidad

9.2.- Otros escenarios de calidad (propuestos)

Escenario	Contexto / estímulo	Respuesta y criterio medible
Integridad de duplicados	Administrador intenta asignar el mismo revisor dos veces.	Segunda operación no inserta nueva fila; no envía notificación nueva.
Continuidad local	Se interrumpe SMTP al asignar revisor.	La asignación permanece consultable y el registro de correo queda PENDIENTE.
Usabilidad	Participante ejecuta flujo de registro o asignación por primera vez.	≥ 80 % de tareas completas; tiempo y errores anotados.
Rendimiento de consulta	Se cargan 100 preguntas locales en un equipo de referencia.	Pantalla utilizable en ≤ 3 s

Escenario	Contexto / estímulo	Respuesta y criterio medible
Reproducibilidad	Se clona el repo en un equipo limpio con Java 21 y Maven.	mvn clean test finaliza y la UI Swing abre sin configuración especial salvo entorno gráfico.

Tabla 9.2. Escenarios complementarios de calidad

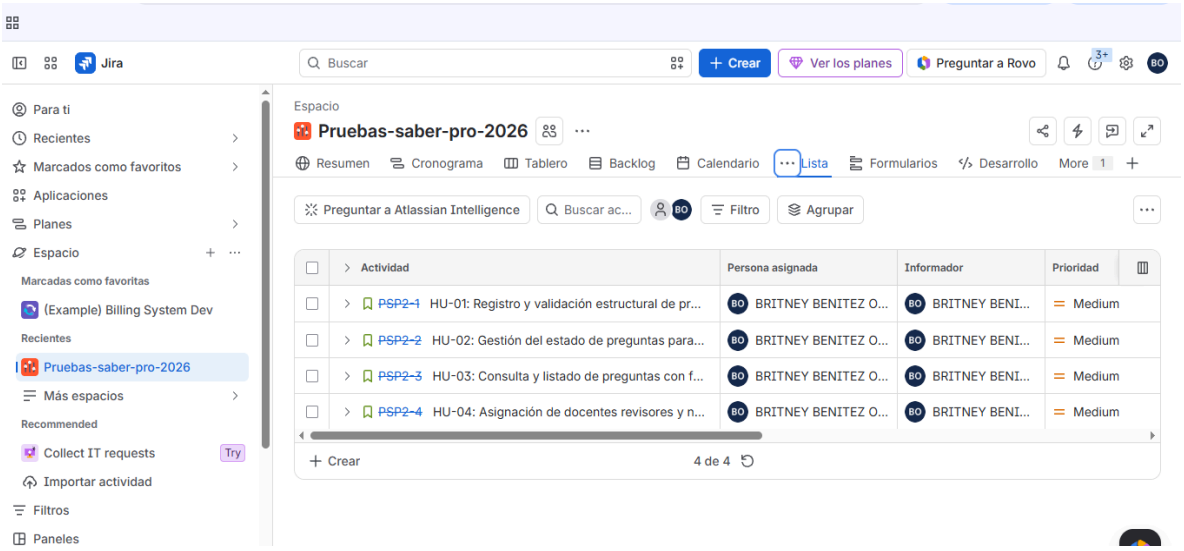
9.3.- Plan de verificación de atributos de calidad

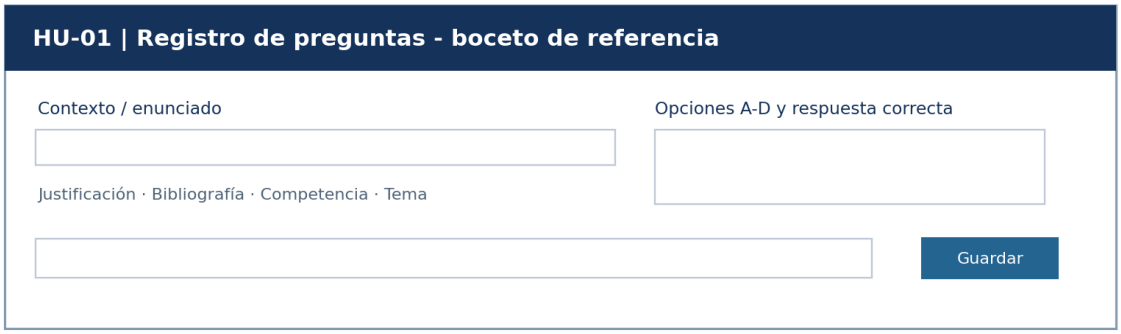
Para verificar los requisitos no funcionales definidos en el proyecto Banco de Preguntas Saber Pro, se establecen los siguientes procedimientos de evaluación. Las métricas indicadas corresponden a criterios de aceptación propuestos y no a resultados experimentales obtenidos.

Requisito	Procedimiento de verificación	Resultado esperado	Estado
RNF-08. Usabilidad	Observar a participantes externos durante las cinco tareas funcionales de las historias de usuario HU-01 a HU-04.	Al menos el 80 % de las tareas completadas sin ayuda.	Evaluación externa pendiente.
RNF-09. Tiempo de respuesta	Ejecutar consultas sobre una base local con 100 preguntas y registrar el tiempo de respuesta.	Tiempo de consulta menor o igual a 3 segundos.	Medición pendiente.
RNF-01. Modificabilidad	Sustituir la configuración del proveedor SMTP y comprobar que el servicio de asignación mantiene su comportamiento y las pruebas automatizadas continúan pasando.	Cambio localizado, sin modificar las reglas de negocio ni introducir regresiones.	Ensayo controlado pendiente.

Conclusión: La arquitectura implementada incorpora mecanismos orientados a la modificabilidad, tales como interfaces, inyección de dependencias y separación de responsabilidades. Los procedimientos descritos permitirán verificar los atributos de calidad mediante mediciones controladas en futuras evaluaciones. No se presentan como resultados medidos las metas que todavía no han sido comprobadas.

10.- Prototipos y evaluación de usabilidad





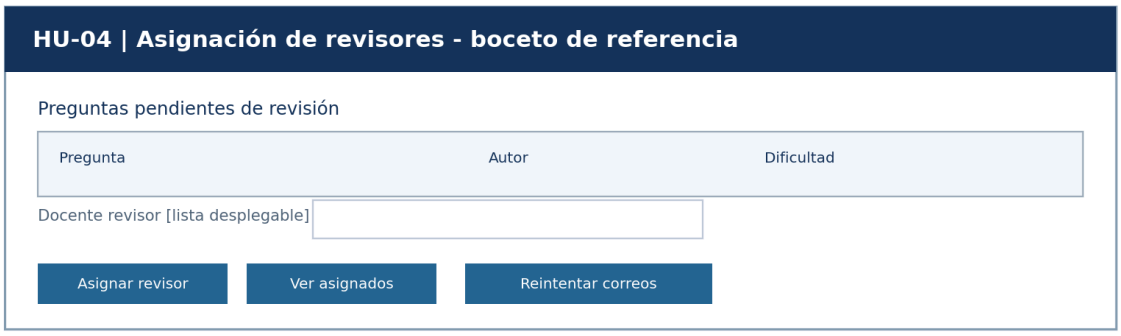


Figura 10.1. Captura del proyecto en Jira y bocetos referenciales de las interfaces HU-01 y HU-04; no constituyen una prueba de usabilidad.

10.1.- Inventario de pantallas a evidenciar

Las interfaces siguientes corresponden a clases que existen en src/main/java/co/edu/unicauca/saberpro/presentation/view. Se describe el comportamiento implementado, no las pantallas de autenticación y edición de preguntas del documento ajeno que se utilizó como referencia visual. Los bocetos anteriores son referenciales y no sustituyen capturas de la aplicación funcionando.

INTERFAZ	HU	DESCRIPCIÓN DE FUNCIONALIDAD REAL
Selector de perfiles	General	La ventana inicial permite elegir Autor (HU-01 a HU-03) o Administrador (HU-04). Los perfiles son demostrativos; NO hay uiLogin, registro de cuentas ni autenticación por contraseña.
Registro de preguntas	HU-01	RegistroPreguntaView presenta contexto, enunciado, cuatro distractores, respuesta correcta, justificación, bibliografía, competencia, tema, subtema y dificultad; muestra validaciones y permite guardar en BORRADOR.
Envío a revisión	HU-02	PreguntaEstadoView consulta por identificador, muestra el estado y permite la transición autorizada de BORRADOR a PENDIENTE_REVISION.
Mis preguntas	HU-03	ConsultaPreguntaView lista solo las preguntas del autor, filtra por estado/dificultad, pagina resultados y permite abrir detalles.
Registro de docentes	HU-04	RegistroRevisorView registra ID, nombre y correo del docente; valida datos y rechaza duplicados.
Asignación de revisores	HU-04	AsignacionRevisionView muestra preguntas pendientes y docentes registrados; permite asignar, ver asignaciones, consultar detalles y evitar el mismo par duplicado.
Notificaciones y reintentos	HU-04	El administrador usa Reintentar notificaciones para procesar correos pendientes y visualizar procesadas, enviadas y fallidas; SMTP se configura por variables de entorno.

Tabla 10.1. Inventario de pantallas a evidenciar

10.2.- Protocolo de prueba de usabilidad

Objetivo: medir la facilidad de uso de las tareas principales con personas diferentes de la desarrolladora. Se propone invitar a tres participantes, obtener su consentimiento y observarlos individualmente sin guiar la ejecución.

Estado actual: la desarrolladora realizó una autoevaluación funcional de T1 a T5. No se efectuó todavía la sesión independiente propuesta con tres participantes; por lo tanto, las metas de tiempo de la Tabla 10.2 NO son mediciones obtenidas.

ID	OBJETIVO	DESCRIPCIÓN DE LA TAREA	DURACIÓN ESTIMADA	MATERIALES
UT-01	Comprender el registro de preguntas.	Desde Autor, crear una pregunta válida y luego intentar registrar otra con un distractor repetido; observar mensajes y errores.	4–6 min (meta)	Aplicación Swing; datos de prueba; ficha de observación.
UT-02	Comprender el cambio de estado.	Localizar una pregunta propia en BORRADOR, enviar a revisión e identificar el nuevo estado y su color.	2–4 min (meta)	Pregunta registrada; pantalla HU-02.
UT-03	Localizar preguntas eficientemente.	Entrar en Mis preguntas; aplicar estado y dificultad, cambiar página y consultar los detalles de un registro.	3–5 min (meta)	Base con varias preguntas; pantalla HU-03.
UT-04	Asignar correctamente un docente.	Desde Administrador, registrar un revisor y asignarlo a una pregunta pendiente. Probar una segunda asignación idéntica.	4–6 min (meta)	Pregunta PENDIENTE_REVISION; correo de prueba; pantallas HU-04.

ID	OBJETIVO	DESCRIPCIÓN DE LA TAREA	DURACIÓN ESTIMADA	MATERIALES
UT-05	Interpretar fallo y reintento de correo.	Con SMTP de prueba deshabilitado, identificar notificación pendiente y localizar Reintentar notificaciones; no usar credenciales personales de terceros.	2–4 min (meta)	Entorno de prueba sin SMTP; listado de notificaciones.

Tabla 10.2. Tareas del test de usabilidad

Registro de la ejecución: anotar por participante y tarea el tiempo real, si completó sin ayuda, número de errores, dudas verbales y satisfacción en escala de 1 a 5. La tasa de éxito se calculará solo con respuestas reales. Las duraciones estimadas de la tabla no son tiempos observados.

10.3.- Instrumento de recolección

Participante	Tarea	¿Completó?
Britney Benítez Ordoñez	Registrar una pregunta Saber Pro con sus validaciones (HU-01).	Sí
Britney Benítez Ordoñez	Cambiar una pregunta de BORRADOR a PENDIENTE_REVISION (HU-02).	Sí
Britney Benítez Ordoñez	Consultar preguntas con filtros, paginación y detalles (HU-03).	Sí
Britney Benítez Ordoñez	Registrar docentes y asignarlos a preguntas pendientes (HU-04).	Sí
Britney Benítez Ordoñez	Enviar una notificación por correo electrónico y comprobar su recepción (HU-04).	Sí

Tabla 10.3. Registro individual de tareas completadas en autoevaluación

Registro de autoevaluación (una sola desarrolladora): las cinco funciones se marcaron como completadas. El resultado «Sí» se refiere a comprobación funcional, no a una tasa de usabilidad de usuarios externos.

10.3.1. Simulación exploratoria de experiencia de usuario

Objetivo de la simulación

Analizar de manera exploratoria la facilidad de uso de las funcionalidades del sistema Banco de Preguntas Saber Pro, considerando la perspectiva de una persona que utiliza la aplicación por primera vez..

Perfil del usuario simulado

- Identificador: P-SIM01.
- Tipo de participante: Usuario Juan Carlos Velez
- Conocimientos informáticos: Básicos.
- Objetivo: Registrar, consultar y gestionar preguntas Saber Pro sin conocer previamente la organización de las pantallas.

Resultados de la experiencia

Tarea	Tiempo simulado	Resultado	Posible dificultad	Facilidad estimada (1–5)
T1 – HU-01. Registrar pregunta Saber Pro	5 min 20 s	Completada	El usuario omite inicialmente el campo de justificación y debe corregirlo antes de guardar.	4
T2 – HU-02. Enviar pregunta a revisión	2 min 50 s	Completada	El usuario necesita identificar el código de la pregunta para consultar su estado y enviarla a revisión.	4
T3 – HU-03. Consultar preguntas	4 min 10 s	Completada	El usuario explora los filtros por estado y dificultad antes de localizar los controles de paginación.	3
T4 – HU-04. Asignar docente revisor	5 min 45 s	Completada	El usuario desconoce inicialmente que la pregunta debe encontrarse en PENDIENTE_REVISION antes de asignar un docente.	3
T5 – HU-04. Interpretar y reintentar notificaciones	3 min 30 s	Completada	El usuario necesita comprender la diferencia entre PENDIENTE y ENVIADA e identificar la opción de reintento.	3

Análisis de la experiencia

El ejercicio plantea que una persona sin experiencia previa podría comprender las operaciones principales del Banco de Preguntas Saber Pro, aunque algunas tareas requieren conocer las reglas de negocio y los estados utilizados por el sistema.

Las principales oportunidades de mejora identificadas hipotéticamente se relacionan con la claridad de los campos obligatorios, la identificación de preguntas mediante su código, la navegación por resultados y la explicación del procedimiento de asignación de revisores.

En particular, la asignación de docentes revisores requiere que el usuario comprenda la relación entre HU-02 y HU-04, debido a que las preguntas deben enviarse a revisión antes de ser asignadas.

Recomendaciones de mejora

1. Incorporar indicaciones visuales para distinguir los campos obligatorios.
2. Facilitar la identificación del código generado al registrar una pregunta.
3. Mejorar la visibilidad de los botones de navegación y paginación.
4. Agregar instrucciones breves para explicar la relación entre el envío a revisión y la asignación de docentes.
5. Explicar de forma sencilla los estados de las notificaciones y el procedimiento de reintento.

10.4.- Hallazgos y mejoras

Aspecto identificado en autoevaluación	Severidad	Recomendación	Responsable / fecha	Estado
HU-01 a HU-04: La aplicación utiliza una selección manual de los perfiles Autor y Administrador, sin un sistema de autenticación real. Esto puede generar confusión respecto al alcance de los permisos durante la demostración.	Baja	Incorporar un mensaje que indique que la selección de perfiles es demostrativa y mostrar claramente el perfil activo.	Britney Benítez Ordoñez / 13-10-2026	Mejora propuesta
HU-04: Para asignar un docente revisor, primero debe existir una pregunta registrada y enviada al estado PENDIENTE_REVISION. Esta dependencia puede dificultar la comprensión del procedimiento para alguien que utiliza el sistema por primera vez.	Media	Incorporar instrucciones breves que expliquen los pasos necesarios antes de realizar la asignación de revisores.	Britney Benítez Ordoñez / 13-10-2026	Mejora propuesta

Tabla 10.4. Aspectos de autoevaluación y mejoras propuestas

Observación: La revisión fue realizada por la desarrolladora del sistema. Los aspectos registrados corresponden a características conocidas de la aplicación y a posibles oportunidades de mejora de la experiencia de usuario. Las severidades son estimadas y no se dispone de resultados de pruebas realizadas con usuarios externos.

11.- Pruebas y matriz de trazabilidad

11.1.- Evidencia técnica de pruebas

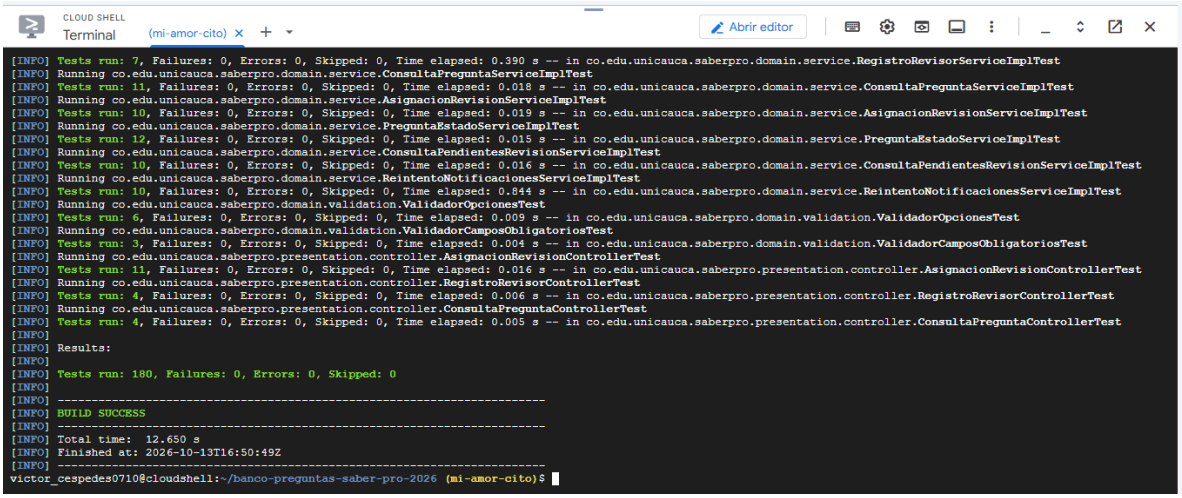
Verificación automatizada: en la última ejecución comunicada desde Google Cloud Shell se obtuvieron 180 pruebas JUnit 5 exitosas, sin fallos ni errores; después de retirar un archivo Java duplicado, la desarrolladora volvió a comprobar las 180 pruebas. La siguiente tabla vincula suites reales del repositorio con objetivos y artefactos; son pruebas unitarias/integración, NO pruebas de usabilidad. Las duraciones deben registrarse mediante Maven y no se inventan.

Evidencia complementaria requerida para la entrega: incorporar una captura o registro de la última ejecución de mvn clean test con su resumen de pruebas y BUILD SUCCESS. El documento no incorpora actualmente ese pantallazo.

ID	OBJETIVO	DESCRIPCIÓN Y COMPROBACIÓN	DURACIÓN	ARTEFACTOS DEL PROYECTO
PU-01	Validar registro de preguntas	Comprobar entidad, campos obligatorios, distractores y rechazo de entradas incorrectas.	Medir en Maven	PreguntaTest; PreguntaServiceImplTest; ValidadorOpcionesTest.
PU-02	Validar envío a revisión	Probar rol, propiedad de la pregunta y cambio BORRADOR → PENDIENTE_REVISION.	Medir en Maven	PreguntaEstadoServiceImplTest; PreguntaCambioEstadoTest.

ID	OBJETIVO	DESCRIPCIÓN Y COMPROBACIÓN	DURACIÓN	ARTEFACTOS DEL PROYECTO
PU-03	Validar consultas y filtros	Comprobar filtro por autor/estado/dificultad, paginación, conteos y controlador.	Medir en Maven	ConsultaPreguntaServiceImplTest; ConsultaPreguntaControllerTest.
PU-04	Registrar docentes revisores	Validar alta de revisor, restricciones de rol e ID/correo duplicados.	Medir en Maven	RegistroRevisorServiceImplTest; RegistroRevisorControllerTest.
PU-05	Controlar asignaciones	Probar pregunta pendiente, rol administrador, múltiples revisores y duplicados.	Medir en Maven	AsignacionRevisionServiceImplTest; SqliteAsignacionRevisionRepository Test.
PU-06	Persistir y consultar SQLite	Verificar estados, datos, asignaciones y notificaciones bajo repositorios de prueba.	Medir en Maven	SqlitePreguntaRepositoryTest; SqliteNotificacionRevisionRepositor yTest.
PU-07	Comprobar correo y reintentos	Probar notificador desacoplado, registro de PENDIENTE/ENVIADA y reintentos sin credenciales reales.	Medir en Maven	NotificadorRevisionEmailTest; ReintentoNotificacionesServiceImpl Test.
PF-08	Verificación SMTP manual	Registrar notificación y comprobar correo recibido utilizando remitente y destinatario autorizados; no automatizado.	Tiempo real no registrado	Evidencia de aplicación: 1 procesada, 1 enviada, 0 fallidas; captura de recepción.

Tabla 11.1. Evidencias técnicas de prueba



11.2.- Casos de prueba funcionales mínimos

Prueba	Caso	Entrada / situación	Resultado esperado
CP-01	CU-01	Campos completos	Se guarda BORRADOR
CP-02	CU-01	Opción repetida	Validación impide guardar
CP-03	CU-02	Autor y pregunta propia	Cambio a PENDIENTE_REVISION

Prueba	Caso	Entrada / situación	Resultado esperado
CP-04	CU-02	Pregunta de otro autor	Acceso denegado
CP-05	CU-03	Estado + dificultad	Filtrado correcto
CP-06	CU-03	Más de 10 registros	Paginación correcta
CP-07	CU-04	Revisor válido	Registro en SQLite
CP-08	CU-04	Correo duplicado	Registro rechazado
CP-09	CU-06	Revisor nuevo a pendiente	Asignación registrada
CP-10	CU-06	Asignación duplicada	No se crea duplicado
CP-11	CU-06	SMTP no configurado	Asignación queda y notificación PENDIENTE
CP-12	CU-08	SMTP funcional tras fallo	PENDIENTE cambia a ENVIADA
CP-13	CU-07	Pregunta asignada	Listado muestra su revisor
CP-14	Transversal	Perfil inadecuado	Servicio deniega operación

Tabla 11.2. Casos de prueba funcionales

11.3.- Trazabilidad HU-RF-CU-prueba

Historia	Requisitos asociados	Casos de uso	Pruebas de referencia
HU-01	RF-01–RF-04	CU-01	CP-01, CP-02; PreguntaServiceImplTest y validadores.
HU-02	RF-05–RF-07	CU-02	CP-03, CP-04; PreguntaEstadoServiceImplTest.
HU-03	RF-08–RF-11	CU-03	CP-05, CP-06; ConsultaPreguntaServiceImplTest.
HU-04	RF-12–RF-20	CU-04–CU-08	CP-07–CP-13; asignación, notificación y SQLite.
Transversal	RF-21–RF-22	Todos	CP-14, pruebas de persistencia y rol.

Tabla 11.3. Trazabilidad de historias y pruebas

12.- Gestión ágil del proyecto y contribuciones

12.1.- Backlog académico y seguimiento de los sprints

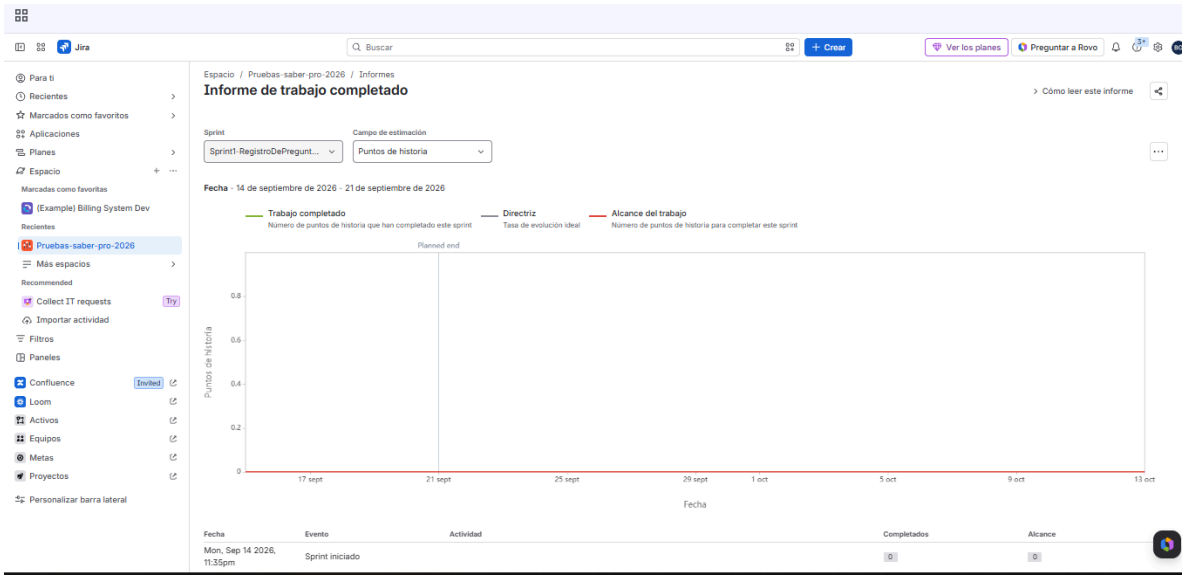
La guía del curso exige una captura real del Sprint 1 en Jira o Trello. El tablero debe reflejar las tareas efectivamente realizadas por la única desarrolladora, con responsables y estados verificables. La siguiente tabla resume evidencias de las cuatro historias distribuidas en cuatro sprints; no debe confundirse con la planificación histórica exclusiva del Sprint 1. Las subtareas fueron registradas después del cierre de los sprints.

Elemento backlog	HU / tarea	Evidencia de repositorio	Asignado a	Estado tablero real
BL-01	HU-01: registrar y validar	Jira PSP2-1 / PR #1	Britney Benítez Ordoñez	Listo en Jira
BL-02	HU-02: enviar a revisión	Jira PSP2-2 / PR #2	Britney Benítez Ordoñez	Listo en Jira
BL-03	HU-03: filtros y consulta	Jira PSP2-3 / PR #3	Britney Benítez Ordoñez	Listo en Jira
BL-04	HU-04: asignaciones y SMTP	Jira PSP2-4 / PR #4 y #7	Britney Benítez Ordoñez	Listo en Jira
BL-05	README / ejecución	PR #5 y #6	Britney Benítez Ordoñez	Documentación integrada
BL-06	Arquitectura / C4 / UML	Documento de arquitectura del proyecto	Britney Benítez Ordoñez	Completado
BL-07	Usabilidad / prototipos	Secciones 10.1 a 10.4 del documento	Britney Benítez Ordoñez	Autoevaluación documentada; prueba externa pendiente
BL-08	Video de sustentación	Carpeta Drive indicada en sección 13	Britney Benítez Ordoñez	Enlace Drive consignado; YouTube por verificar

Tabla 12.1. Backlog académico y seguimiento de las cuatro historias

12.2.- Evidencia de Jira y trazabilidad de Sprint 1

Captura real de la vista «Lista» del proyecto PSP2, con HU-01 desplegada y sus cinco subtareas PSP2-5 a PSP2-9. Las actividades fueron creadas con posterioridad al cierre del sprint; por tanto, esta captura muestra trazabilidad de trabajo, NO la planificación original del Sprint 1.



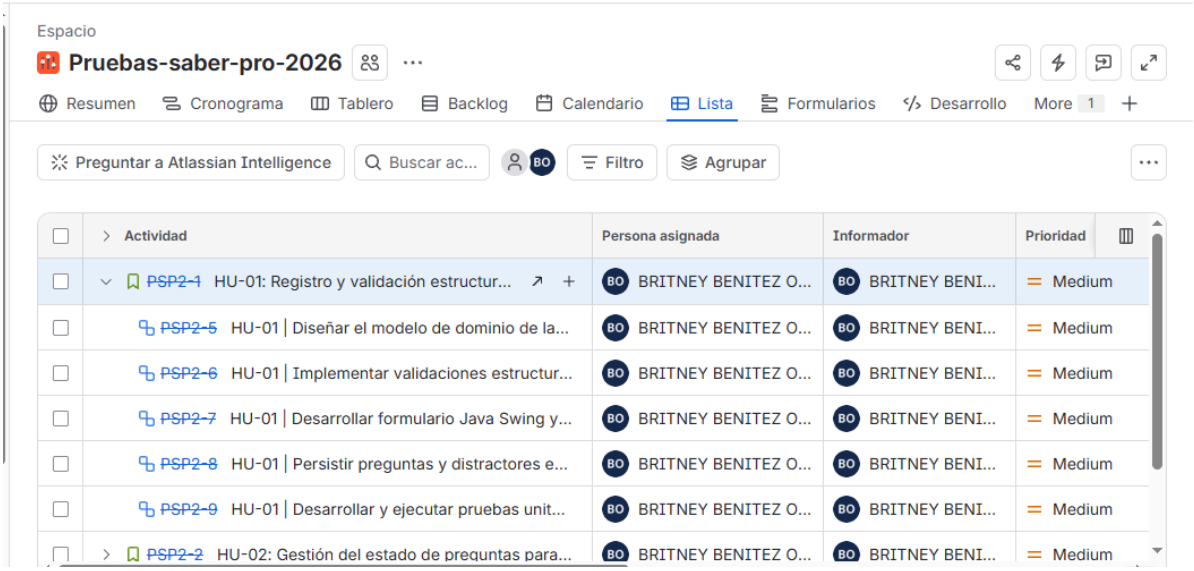


Figura 12.1. Vista real de Jira: historia HU-01 y cinco subtareas vinculadas en la pestaña Lista.

Sprint1-RegistroDePreguntas: estado Cerrado en Jira (13 de octubre de 2026); historia PSP2-1 en estado Listo. Las 21 subtareas del proyecto también están en estado Listo. Para reconstruir la planificación histórica, consultar el informe del sprint cerrado; el listado actual no constituye una captura de planificación previa.

LINK JIRA

https://banco-preguntas-unicauca.atlassian.net/jira/software/projects/PSP2

12.3.- Contribuciones verificables de la desarrolladora

Este proyecto fue desarrollado individualmente por Britney Benítez Ordoñez. La autora realizó análisis de requisitos, diseño de arquitectura en tres capas, implementación de HU-01 a HU-04, interfaz Swing, SQLite, correo SMTP, pruebas JUnit 5 y documentación técnica. El historial de commits y Pull Requests del repositorio proporciona evidencias verificables de esas actividades. No se atribuyen aportes a integrantes inexistentes. La guía académica contempla grupos de 3 a 5 estudiantes; la modalidad individual debe confirmarse con la docente antes de la entrega.

Desarrolladora	Rol / responsabilidad real	Evidencia (commit/PR/documento)	Fecha
Britney Benítez Ordoñez	Análisis de requisitos, diseño de arquitectura MVC y tres capas, desarrollo de HU-01 a HU-04, implementación Java Swing, SQLite, SMTP, pruebas JUnit 5 y documentación técnica.	Repositorio GitHub y Pull Requests #1 al #7.	Octubre de 2026

Tabla 12.2. Contribuciones individuales verificables

Repositorio: https://github.com/britneybenitez-gif/banco-preguntas-saber-pro-2026

Pull Requests integrados verificados durante revisión de repositorio: #1 HU-01, #2 HU-02, #3 HU-03, #4 HU-04, #5–#6 documentación, #7 corrección del controlador duplicado. El estado y la lista pueden cambiar tras nuevas aportaciones.

13.- Guion de sustentación (12 minutos)

Presentación propuesta para una sola desarrolladora, sujeta a autorización de la docente porque la guía original prevé equipos. La duración y los bloques siguen el protocolo académico de 12 minutos; los resultados de usabilidad se expondrán únicamente cuando existan datos reales.

Bloque	Duración	Qué demostrar	Responsable
Historias, CA, prototipos y usabilidad	2 min	HU-01 a HU-04, criterios, tareas y resultados reales del test.	Britney Benítez Ordoñez
Atributos de calidad	1 min	Escenario de modificabilidad y medición.	Britney Benítez Ordoñez
Arquitectura C4 y UML	2 min	C4 contexto, contenedores, componentes, caso de uso y secuencia.	Britney Benítez Ordoñez
Pruebas unitarias	1 min	mvn clean test: suite completa y ejemplos clave.	Britney Benítez Ordoñez
Git + tablero Sprint 1	1 min	Contribuciones de cada integrante y tablero de tareas.	Britney Benítez Ordoñez

Bloque	Duración	Qué demostrar	Responsable
Demo software y código	5 min	HU-01 → HU-04; SQLite, Decorator, notificación y reintento.	Britney Benítez Ordoñez

Tabla 13.1. Guion de sustentación

Enlace de carpeta de Google Drive aportado para la sustentación (verificar que contenga el video y permita acceso): https://drive.google.com/drive/folders/1_5HzJgT9fSfKCVkjcS_2WyRDT6QKCIa8?usp=sharing

13.1.- Secuencia de demostración funcional sugerida

1. Clonar o abrir el proyecto; explicar JDK21 y Maven y mostrar inicio con perfil Autor.
2. Registrar pregunta completa HU-01 y mostrar su estado BORRADOR.
3. Buscarla por identificador y enviarla a PENDIENTE_REVISION en HU-02.
4. Consultar con filtros y paginación de HU-03.
5. Reiniciar con perfil Administrador y registrar docente revisor HU-04.
6. Asignar revisor y demostrar que una segunda asignación del mismo par es rechazada.
7. Mostrar estado ENVIADA o, si no hay SMTP, PENDIENTE y botón de reintento; no revelar secretos.
8. Mostrar componentes relevantes y pruebas; cerrar con limitaciones y próximos pasos.

14.- Instalación, operación y seguridad de entrega

14.1.- Entorno de ejecución

Dependencia	Requisito
Java	JDK 21
Construcción	Apache Maven
IDE	Visual Studio Code u otro con soporte Maven
Interfaz	Entorno gráfico que permita Java Swing
Persistencia	SQLite JDBC, librería descargada por Maven
Correo	Servidor SMTP con STARTTLS, si se desea el envío real

Tabla 14.1. Dependencias de ejecución

14.2.- Operaciones

1. Clonar el repositorio: git clone https://github.com/britneybenitez-gif/banco-preguntas-saber-pro-2026.git
2. Entrar en su carpeta: cd banco-preguntas-saber-pro-2026
3. Ejecutar prueba completa: mvn clean test
4. Iniciar aplicación: mvn exec:java -Dexec.mainClass=co.edu.unicauca.saberpro.Main
5. Probar recorrido: Autor HU-01 → HU-02 → HU-03; reiniciar y elegir Administrador HU-04.

14.3.- Seguridad de correo y política de entrega

Las variables SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASSWORD y SMTP_FROM se leen desde System.getenv. Para Gmail puede usarse smtp.gmail.com:587 con contraseña de aplicación si está disponible. No se incluye ninguna contraseña real ni credencial personal en este documento, el repositorio o los diagramas.

El docente evaluador debe configurar sus credenciales SMTP propias o utilizar la demostración/grabación realizada por la desarrolladora. Si SMTP está ausente, la asignación persiste y el correo queda PENDIENTE, sin simular un envío exitoso. El sistema no utiliza ni necesita claves para compilar y ejecutar pruebas unitarias.

14.4.- Control de versiones y ramas

Rama principal protegida: main. Ramas de historia: feature/hu-01, feature/hu-02, feature/hu-03 y feature/hu-04. El flujo recomendado es commit en rama, Pull Request, revisión y merge. Este documento no modifica el código ni crea ramas adicionales.

15.- Bibliografía, fuentes y enlaces

15.1.- Fuentes y enlaces

- Guía oficial del primer corte de Ingeniería de Software II, Universidad del Cauca (2026.2).
- Repositorio del proyecto: https://github.com/britneybenitez-gif/banco-preguntas-saber-pro-2026
- Proyecto Jira: https://github.com/britneybenitez-gif/banco-preguntas-saber-pro-2026
- Link video proyecto: https://drive.google.com/drive/folders/1_5HzJgT9fSfKCVkjcS_2WyRDT6QKCIa8?usp=sharing
- README: archivo README.md disponible en la rama main del repositorio.
- Pull Requests: consultar la pestaña Pull requests del repositorio en GitHub.

Bibliografía

[1] Universidad del Cauca, «Entregables proyecto de curso — Primer Corte 2026.2», guía académica proporcionada para Ingeniería de Software II, 2026.

[2] Universidad del Cauca, «Formato-Documento», plantilla institucional de presentación y normas editoriales, suministrada por el equipo, 2026.

[3] Oracle, «Java Platform, Standard Edition 21 API Specification». [En línea]. Disponible: https://docs.oracle.com/en/java/javase/21/docs/api/

[4] JUnit Team, «JUnit 5 User Guide». [En línea]. Disponible: https://junit.org/junit5/docs/current/user-guide/

[5] SQLite, «SQLite Documentation». [En línea]. Disponible: https://www.sqlite.org/docs.html